Appendix 2: A Decision Model for Blockchain-Based Design Patterns Based on Quality Attributes

We present a comprehensive decision model to aid in the selection of blockchain design patterns based on quality attributes. By means of systematic literature review (SLR) and expert interviews, and through iterative optimization, we extracted 72 distinct patterns from a collection of 179 design patterns, identified 28 typical challenges, 13 concerns to address, and 18 related quality attributes. We organized them based on the blockchain's six-layer architecture. We also identified the challenges addressed by these patterns in each layer. This provided developers with a clear understanding of design patterns distribution and their solutions to common challenges. In addition, we established a relationship between design patterns and quality attributes, allowing patterns' impact to be intuitively reflected. By combining different patterns, developers can meet various system development requirements, including FRs and NFRs. Overall, this decision model empowers developers to comprehend how design patterns support blockchain-based applications at each layer. Furthermore, it allows them to make appropriate pattern selections based on quality attributes, simplifying the design and development process.

In this appendix, we introduce the decision model of design patterns and discuss the relationship between design patterns and quality attributes.

Firstly, we introduce the concepts of each layer and describe the concerns within each layer as well as the typical challenges associated with each concern. Subsequently, for each design pattern that addresses the typical challenges, we first introduce its solution, followed by an analysis of its quality attributes and reasons. In some cases, the inherent attributes of the pattern are further discussed. In the end, practical application cases or code samples for each pattern are provided. Finally, to facilitate decision-making, we analyze the commonalities and specificities of the design patterns to illustrate their comparisons and connections. The commonalities among multiple design patterns will be summarized at the end. The numbering of all elements in this section is consistent with Table 1. Due to space constraints, only key concepts are summarized in the pattern descriptions. Readers may refer to the website or references for more detailed information.

Note that for the sake of highlighting and the intuition of the decision model, some of the impacts on quality attributes will only be mentioned in the document, but not labeled in the decision model. It is due to the fact that these effects on quality attributes have a low probability of occurrence and are not significant. Only major and representative impacts on quality attributes (or mapping relationships with design patterns) are labeled in the decision model diagrams. However, this is not a common occurrence. The vast majority of impacts (or mapping results) are already presented in the proposed decision model.

1 Notations

Before introducing, the notation used in our proposed decision model is described by Fig. 1. In general, the main elements consist of "problem", "pattern", and "constraint", which are defined in the figure legend. The lines denote the connections between the various elements.

Solid arrows indicate the path within the decision model, which represents the solution to the challenges in different layers. The concept of "sub-pattern" refers to the subordinate relationship between two patterns resulting from a specialized design for a particular requirement, whether functional or non-functional. The sub-pattern is derived from the originating pattern, indicated by a diamond-shaped arrow. As the sub-pattern is connected to the solid arrow, it is also part of the decision path. The quality attributes of the design pattern are the sum of attributes labeled on the decision model path represented by the solid arrow. The common attributes are labeled before the arrow bifurcation, whereas unique attributes are listed after it, and additional attributes of the sub-pattern are marked individually. Quality attributes marked with the "Δ" tag indicate features compared to those found in traditional systems. And attributes labeled with the "$\sim$" tag are used for internal comparisons between a few patterns within the same group. The remaining objective attributes without tag make up the majority.

The hollow arrow is a notation used to indicate the relationship between different challenges. It points from one challenge to another, and it is labeled [support]. This label denotes a complementary relationship between the two challenges, where addressing the originating problem can provide one-way or both-way help in solving the targeted problem. If the relationship is both-way, the arrow will be bidirectional. Brief information about the relationship will be labeled in the figure. Specifically, the use of design patterns to solve the originating problem can provide certain benefits to the solving of the targeted problem in certain circumstances.

The dotted line without arrows connects pattern and inherent attribute, symbolizing the intrinsic nature or limitations of the pattern. Inherent attributes could be limitations when making a decision, or unique features that set it apart from others.

TABLE 1: Blockchain-based Design Pattern Collection and Layered Taxonomy

Blockchain Layers	Concerns to Address	Typical Challenges	Design Patterns	Alternative Names
L1: Data Layer	C1.1: Data Interaction	C1.1.1: How can outside systems access data on blockchain?	DP1: Pull-Based Outbound Oracle [1]	Reverse Oracle [2], [3], Reverse Verifier [4]
			DP2: Push-Based Outbound Oracle [1]	Anti-Oracle [5]
		C1.1.2: How can blockchains get data from outside?	DP3: Pull-Based Inbound Oracle [1]	Oracle (Data Provider) [6], [7], Oracle [8], Verifier [4], Judge [5]
			DP4: Select-Storage [9]	-
			DP5: Bulletin Board [5]	-
			DP6: Push-Based Inbound Oracle [1]	Oracle [2], [3]
	C1.2: On/Off-Chain Association	C1.2.1: How to reduce both storage and computation costs on chain?	DP7: State Channel [3], [4]	Off-Chain Signatures [10]
		C1.2.2: How to reduce storage costs on chain?	DP8: Off-Chain Datastore [4]	Off-Chain Data Storage [3], Blockchain Anchor [11], Hash Storage [12], Mirror [13], Minimize On-Chain Data [14], Hash Integrity [15], [16], Hash Timestamping [17]
			DP9: Content-Addressable Storage [10]	-
		C1.2.3: How to reduce computation costs on chain?	DP10: Confidential and Pseudo-Anonymous Contract Enforcement [18]	-
			DP11: Delegated Computation [10]	-
	C1.3: Data Migration	C1.3.1: How to extract states from the blockchain?	DP12: Snapshotting [19]	-
		C1.3.2: How to transform states to the target blockchain?	DP13: State Aggregation [19]	-
			DP14: Token Burning [19]	-
		C1.3.3: How to load states from the source blockchain?	DP15: Establish Genesis [19]	-
			DP16: State Initialization [19]	-
			DP17: Node Sync [19]	-
			DP18: Hard Fork [19]	-
			DP19: Exchange Transfer [19]	-
	C1.4: Data Verification	C1.4.1: How to manage on-chain keys?	DP20: Master & Sub Key Generation [11]	-
			DP21: Hot & Cold Wallet Storage [11]	-
			DP22: Key Shards [11]	-
		C1.4.2: How to improve contract data security?	DP23: Commit and Reveal [4], [4], [7]	Commitment [20]
			DP24: Encrypting On-Chain Data [3]	Data Encryption [15], On-Chain Encryption [16], [17]
L2: Incentive Layer	C2.1: Issuance Mechanism	C2.1.1: How to value on-chain assets via token?	DP25: Tokenisation [3]	Token [5], [8], [20]
	C2.2: Distribution Mechanism	C2.2.1: How to incentivize blockchain maintenance?	DP26: Incentive Execution [3], [4]	-
	C3.1: Contract Authorization	C3.1.1: How to realize contract permission control?	DP27: Authorization [5], [8]	Embedded Permission [3], [4], [15], [16], Access Restriction [6], Ownership [7]
		C3.1.2: How to realize contract permission transfers?	DP28: Dynamic Binding [4], [15], [16]	Off-Chain Secret Enabled Dynamic Authorization [3], Hash Secret [21], Ownership and Access Restriction [22]
			DP29: Multiple Authorization [3]	Multi-Signature [21], Multiple Authorities [15], [16]

TABLE 1: Blockchain-based Design Pattern Collection and Layered Taxonomy (Continued)

Blockchain Layers	Concerns to Address	Typical Challenges	Design Patterns	Alternative Names
L3: Contract Layer	C3.2: Contract Cost Optimization	C3.2.1: How to reduce contract costs on blockchain?	DP30: Tight Variable Packing [4]	Short Constant Strings [14]
			DP31: Limit Storage [14]	-
		C3.2.2: How to reduce contract transactions fee of blockchain?	DP32: Use Libraries [14]	-
			DP33: Write Values [14]	-
			DP34: Limit External Calls [14]	-
			DP35: Fewer Functions [14]	Low Contract Footprint [10]
	C3.3: Contract Management & Operation	C3.3.1: How to create contracts efficiently?	DP36: Contract Factory [21]	Factory Contract [3], [4], Child Smart Contract [20]
			DP37: Abstract Factory [23], [24]	-
		C3.3.2: How to execute contracts efficiently?	DP38: Contract Mediator [21]	-
			DP39: Data Segregation [6], [7]	Data Contract [3], [4], [14], Contract Manager [25]
			DP40: Flyweight [4], [23]	-
			DP41: Contract Façade [21]	Façade [20]
			DP42: State Machine [4], [7], [22]	Event-Ordering [26]
			DP43: Publisher-Subscriber [23], [24], [25]	Contract Observer [21]
		C3.3.3: How to upgrade contracts efficiently?	DP44: Proxy Contract [4], [7]	Proxy [14], [23], Contract Relay [6], Database Proxy [25]
			DP45: Contract Register [6], [7]	Name-Service [20], Migration [5], Contract Registry [3], Entity Registry [25]
			DP46: Contract Composer [21]	Satellite [6], [7]
			DP47: Contract Decorator [21]	-
	C3.4: Contract Security	C3.4.1: How to reduce the risk of attack on contracts?	DP48: Contract Balance Limit [4]	Balance Limit [7], [27]
			DP49: Math [8]	Overflow/Underflow [20]
			DP50: Error-Handling [20]	-
		C3.4.2: How to prevent double-spend attacks?	DP51: Check-Effect-Interaction [4], [6], [7]	-
		C3.4.3: how to prevent re-entrancy attacks?	DP52: Pull Payment [6], [7]	Pull [24]
			DP53: Mutex [4], [4], [7], [20], [27]	Locking [26], Guarded Update [25]
			DP54: Replay-Protection [24]	-
		C3.4.4: How to prevent DoS attacks?	DP55: Speed Bump [7], [27]	-
			DP56: Rate Limit [7], [27]	-
		C3.4.5: How to terminate contracts?	DP57: Automatic Deprecation [6], [7]	Termination [8], Deactivation [24],
			DP58: Emergency Stop [4], [27]	Mortal [7]
	C3.5: Contract Migration	C3.5.1: How to run smart contracts on another blockchain?	DP59: Virtual Machine Emulation [19]	-
			DP60: Smart Contract Translation [19]	-
L4: Application Layer	C4.1: Identity Management & Verification	C4.1.1: How to manage the user identity properly?	DP61: Identifier Registry [11]	-
			DP62: Bound with Social Media [11]	-
			DP63: Dual Resolution [11]	-
			DP64: Delegate List [11]	-
			DP65: Address Mapping [5]	-
			DP66: Multiple Registration [11]	-

TABLE 1: Blockchain-based Design Pattern Collection and Layered Taxonomy (Continued)

Blockchain Layers	Concerns to Address	Typical Challenges	Design Patterns	Alternative Names
		C4.1.2: How to verify the user identity securely?	DP67: Selective Content Generation [11]	-
			DP68: Time-Constrained Access [11]	-
			DP69: One-Off Access [11]	-
	C4.2: Business Cooperation	C4.2.1: How to coordinate entity cooperation?	DP70: Digital Record [13]	Blockchain BP Engine [12], Proof of Integrity [28]
			DP71: Blockchain-Based Reputation System [12]	-
			DP72: Vote [5]	Poll [8]

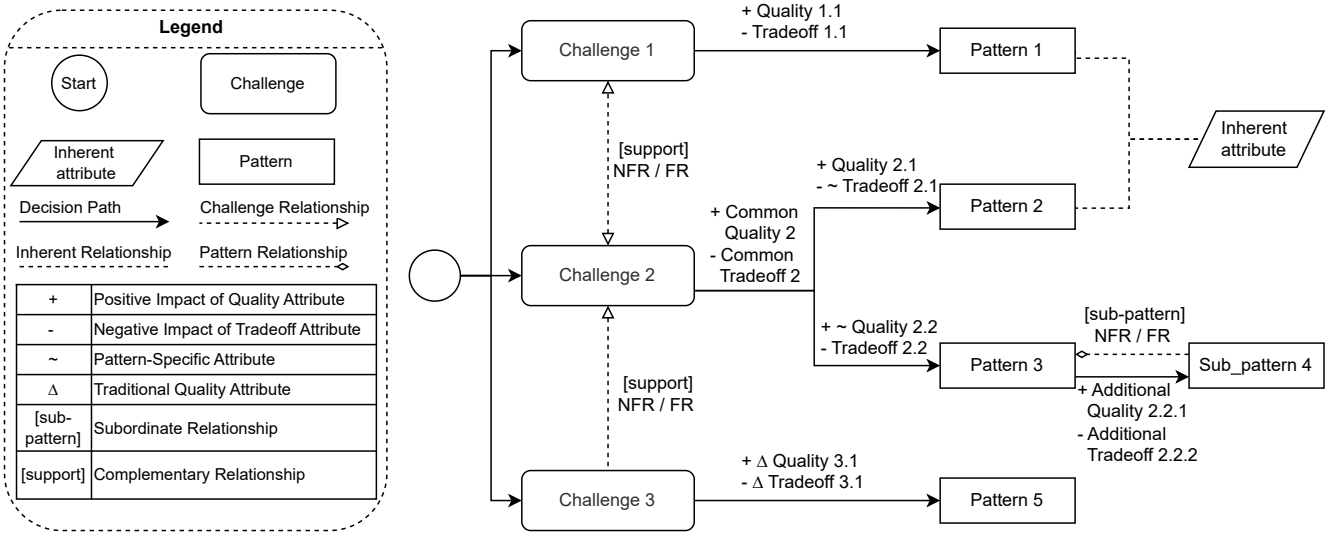

Fig. 1: Notation of the Decision Model

2 Quality Attribute Metrics

Based on ISO and IEEE standards, we have listed the quality attributes metrics and provided descriptions in the context of blockchain, as shown in Table 2. ISO and IEEE standards outline most of the quality attributes identified. Due to the unique properties of blockchain technology, new attributes that are not included in the IEEE and ISO guidelines have been introduced. These attributes will be described separately below:

Transparency has been introduced due to the decentralized nature of blockchain. Any data or operation that is centralized or non-public will decrease data transparency (also recognized as credibility). Transparency does not conflict with privacy. There are some technologies that can ensure both data privacy and transparency, such as zero-knowledge proofs [37].

Data Verifiability has been introduced due to the need for all blockchain validators to record and verify on-chain data for its effectiveness. The introduction of off-chain data may compromise the verifiability because off-chain data sources cannot be tracked and validated. Thus, this attribute describes the degree to which on-chain data can be verified and tracked.

Data Consistency has been introduced because blockchain is a deterministic state machine, and consistency is crucial for its internal consensus and data synchronization. Therefore, it is necessary to evaluate the impact of certain behaviors on data consistency, such as data migration and on/off-chain storage.

Fee Efficiency has been introduced due to the unique incentive mechanisms of blockchain, which require users to pay for each on-chain transaction according to resource occupation. The design pattern's impact on transaction fees is evaluated using this attribute. This attribute is also called *Resource Management* in [20].

Finally, as blockchain is a resource-intensive system, *Cost Efficiency* and *System Simplification* have also been considered to evaluate the pattern consequences in terms of runtime and static complexity, respectively. Both of them are also referred to as *Code Efficiency* in [20] and [36]. It should be noted that *Cost Efficiency* is not the same as *Fee Efficiency*, due to the fact that not all costs within blockchain will result in a fee consumption.

TABLE 2: Quality Attributes and Their Corresponding Metrics

Quality Attributes	Description
Reliability [29], [30], [31]	Capability of a product to perform its intended functions under stated conditions for a specified period.
Usability [29], [30], [31]	The degree to which the operation of software matches the purpose, and intuition of users, in order to make the system easy to use.
Performance [29], [30], [31], [32]	Capability of a product to meet its specified performance requirements for response time, throughput, and resource utilization.
Availability [29], [30]	The degree to which software remains operable in the presence of system failures.
Scalability [33]	The effort required to make the expected growth in service capacity by increasing hardware and software resources.
Interoperability [29] [30], [31]	The degree to which blockchain can be easily connected with off-chain or other on-chain components and operated.
Security [29], [30], [31]	The degree to which software can detect and prevent information leak, malicious behavior, and system resource destruction.
Privacy [34], [35]	The degree to which data is protected from disclosure, and data owners are able to decide when, how, and to what extent to share their personal information.
Flexibility (or Adaptability) [31], [32]	Capability to adapt to changing environments or requirements without significant modifications.
Reusability [29], [30]	The degree to which code/software components can be reused in applications other than the original application.
Extensibility [29], [30]	The degree of effort required to improve or modify the efficiency or functions of software.
Maintainability [29], [30], [31]	Capability of a product to be modified effectively and efficiently by the intended maintainers, covering corrections, improvements, or adaptations.
Transparency [36]	The degree to which a component or data decentralized. It can also be regarded as data transparency in blockchain system.
Data Verifiability [20]	Capability of a blockchain system to verify and track external input data/transaction data on-chain.
Data Consistency [36]	The degree of completeness of the mapping of associated data between two domains.
Fee Efficiency [20]	The degree to which the cost of transaction fee (or gas) decreases at runtime.
Cost Efficiency [36]	The degree to which the cost of computation or storage of software decreases at runtime.
System Simplification [20], [36]	The degree to which the function structure, invocation dependency, and coupling degree of the software is simplified when dealing with specific functionality.

3 Decision Model for Blockchain-Based Design Patterns

L1: Data Layer

The data layer of blockchain technology encompasses the management of underlying data, including data storage methods, account and state migration, and transaction verification. Given the critical nature of data management within the blockchain, numerous design patterns have been developed within this layer. Although the implementation of the data layer varies across different blockchains, we have developed a decision model for design patterns that addresses the common issues. This layer focuses on patterns related to blockchain data management, including data interaction, on/off-chain association, data migration, and data verification.

C1.1: Data Interaction

As part of a broader software system, blockchain must facilitate data communication with off-chain components [38]. However, due to the closed and deterministic nature of the blockchain, direct interaction between on-chain and off-chain data is not available [39], thus posing challenges (C1.1.1) and (C1.1.2). To address these challenges, a standardized approach to on-chain/off-chain data interaction is necessary, thus summarizing the Data Interaction concern (C1.1). Oracle is a common approach used to facilitate blockchain interaction with external systems [40]. Hence, all Oracle-relevant blockchain-based design patterns (DP1 to DP6) are discussed here.

C1.1.1: How can outside systems access data on the blockchain?

DP1: Pull-based Outbound Oracle. In this pattern, blockchain data is filtered and transferred through Oracle so that it can be accessed by external entities. These off-chain entities (or system components) can use Oracle to query on-chain information using an identifier [3]. The asynchronous query mechanism, which decouples off-chain data requests from on-chain data retrieval [1], allows off-chain users to freely query the required on-chain data, thereby enhancing usability. *However*, the introduction of an Oracle to facilitate interactions with the external world necessitates intrusive changes and transformations to the native system [2], resulting in low reusability. Additionally, data retrieval latency can reduce performance, as the blockchain may take long to respond to requests depending on the block size and information

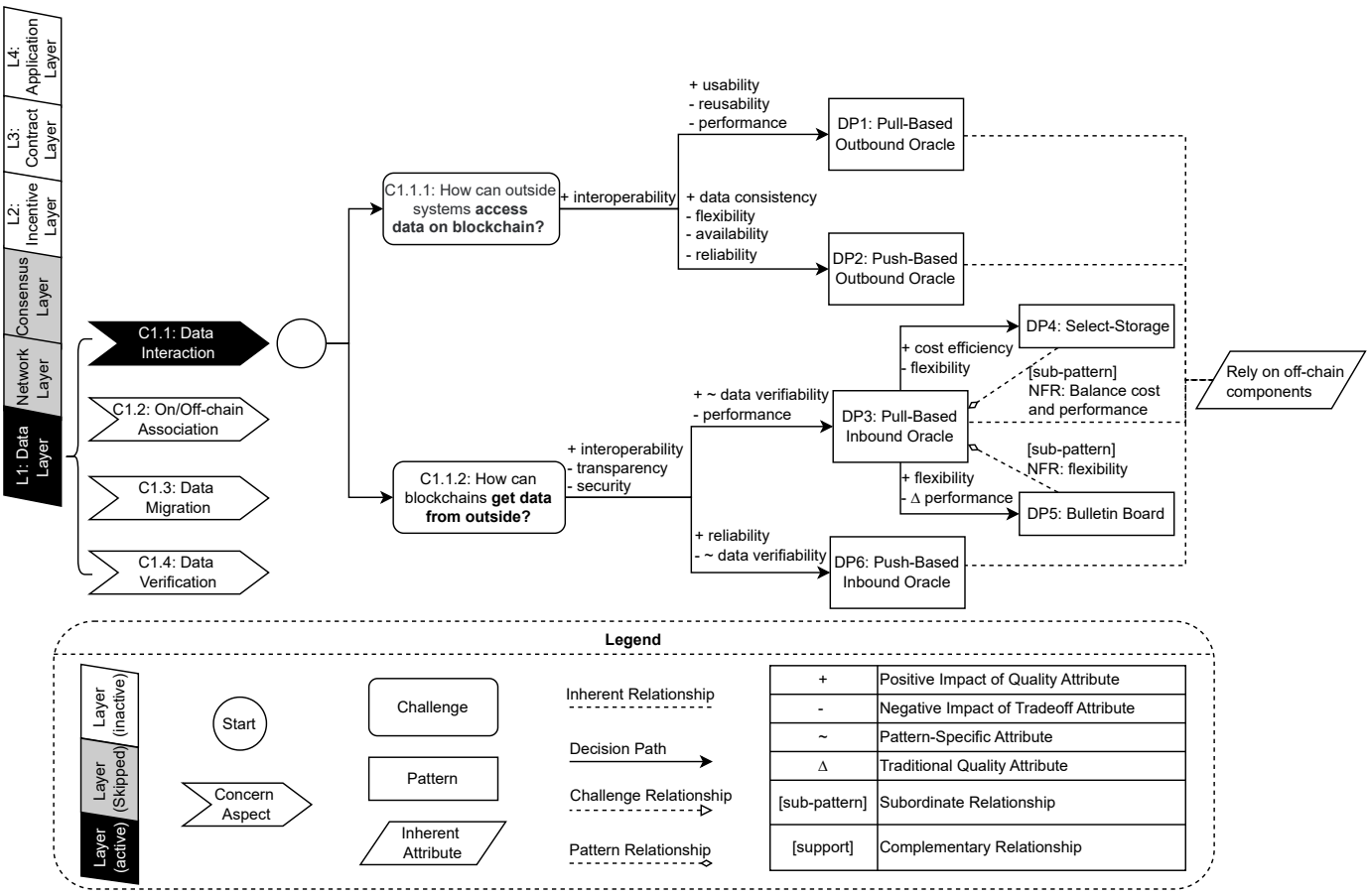

Fig. 2: Decision Model for Data Interaction (C1.1)

location [1]. This pattern has been utilized in Identitii[1] and Slock.it[2]. The ChainLink project [41] also offers examples of its application.

DP2: Push-based Outbound Oracle. In this pattern, the Oracle listens for on-chain state or data changes to initiate off-chain actions. The blockchain event listener is typically an external component implemented in a client [5]. The event monitoring mechanism ensures better data consistency between on-chain and off-chain data. *However*, it lacks flexibility as it can only listen and trigger off-chain activities based on pre-defined events. In addition, an event is normally monitored by only one listener in order to avoid duplicate actions. And the failure of listeners may suspend the off-chain triggering mechanism [1], resulting in interaction blocking and low availability. Moreover, monitoring blockchain events may also introduce uncontrollable delays or crashes, impacting reliability and hindering time-sensitive transactions [1]. There have been several studies exploring the implementation of this pattern, including [1] and [42], which conduct in-depth practice based on this pattern.

C1.1.2: How blockchains get data from outside?

DP3: Pull-based Inbound Oracle. The Oracle retrieves off-chain data upon request from a blockchain application and returns the results via a transaction [1]. This process is transparent with Oracle and prevents contract visibility to all entities [43], [44]. On-chain data is continuously tracked, and it is easy to check the status of off-chain components [6], facilitating the verifiability of off-chain components. However, all information must be actively requested by the blockchain, potentially causing performance bottlenecks and reduced efficiency. Data from off-chain components may also compromise transparency and data credibility. This pattern has also been adopted by Reality Keys [45] and Orisi[1].

DP4: Select-Storage. This pattern resolves the contradiction between the efficiency and cost of data inbound Oracle using a classification model such as SVM (Support Vector Machine). It is the sub-pattern of Pull-based Inbound Oracle. Performance is improved due to the introduction of a classification model, which only puts critical, frequently reused data on-chain [9]. Oracle's transmission efficiency can also be improved. Meanwhile, the caching strategy is dynamically set based on the classification model, balancing frequently used on-chain data for quick response with infrequently used off-chain data for cost-saving, reducing blockchain overhead. However, combining off-chain components and data analysis makes it difficult to deploy. Additionally, training classification models can be time-consuming and computationally expensive, making it difficult to apply them in diverse scenarios and reducing flexibility. Furthermore, security may also

[1]https://identitii.com/
[2]https://slock.it/

TABLE 3: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C1.1: Data Interaction)

ID	Pattern Name	Quality Attribute	Impact	Quality Attribute Mapping Description
DP1	Pull-based Outbound Oracle	Usability	Positive	The asynchronous query mechanism separates off-chain requests from on-chain retrieval, allowing off-chain users to freely query the required on-chain data.
		Reusability	Negative	Introducing an Oracle requires intrusive changes and transformations to the native system.
		Performance	Negative	Data retrieval delays with Oracle can lower performance, depending on block size and data location.
DP2	Push-based Outbound Oracle	Data Consistency	Positive	Event monitoring mechanism ensures better data consistency between on-chain and off-chain data.
		Flexibility	Negative	Limited flexibility due to only predefined events can be triggered by Oracle.
		Availability	Negative	An event is normally monitored by only one listener to avoid duplicate actions, and listener failures can block interactions.
		Reliability	Negative	Monitoring blockchain events may cause delays or crashes.
DP3	Pull-based Inbound Oracle	Data Verifiability	Positive	Continuous tracking of on-chain data enhances verifiability of off-chain components (Compare with DP6).
		Performance	Negative	Performance bottlenecks due to that all information must be actively requested by the blockchain.
DP4	Select-Storage	Cost Efficiency	Positive	caching strategy is dynamically set based on the classification model, which can balance on-chain and off-chain data for cost-saving.
		Flexibility	Negative	Training classification models is time-consuming, and is difficult to be applied in different scenarios.
DP5	Bulletin Board	Flexibility	Positive	Utilizes complex peer-to-peer interactions to ensure flexibility for various scenarios.
		Performance	Negative	Lower throughput compared to other patterns due to dependence on users' responses.
DP6	Push-based Inbound Oracle	Reliability	Positive	Enables reliable external data access through distributed Oracles.
		Data Verifiablility	Negative	The blockchain cannot validate the Oracle data as the Oracle is not triggered by the blockchain (Compare with DP3).
-	Commonality (DP1 to DP6)	Interoperability	Positive	All patterns enhance interoperability by providing methods for interacting with off-chain components, thus improving on-chain and off-chain data exchange.
-	Commonality (DP3 to DP6)	Transparency	Negative	All patterns decrease transparency due to dependency on off-chain components and external data. For inbound Oracles (DP3 to DP6), this is inherent, while for outbound Oracles (DP1 and DP2), it depends on the Oracle's behavior.
-	Commonality (DP3 to DP6)	Security	Negative	These patterns decrease security due to the reliance on external data sources introduced by Oracle that the blockchain cannot verify, leading to potential security vulnerabilities. Note that patterns of outbound Oracles (DP1, DP2) do not compromise security, as Oracles in these patterns only transmit information to external parties.

be compromised due to the potential for tampering and the lack of encryption functions. Due to the low probability and insignificant degree of security threat, the decision model does not include a label for it. A demonstration implementation that utilizes this pattern has been conducted and documented in [9].

DP5: Bulletin Board. This pattern is designed to enable users to post and answer requests in a Q&A-style format through bulletin board contracts, making it a variant of the Pull-based Inbound Oracle pattern. This approach allows users to engage in peer-to-peer interactions that are more complex than simple Oracle queries and can adapt to a variety of dynamic situations [5], and has better flexibility for different and changeable scenarios. However, as a blockchain-based discussion forum, timely results cannot be guaranteed and throughput is lower than other patterns. Additionally, results depend on other users' answers. Hence, if numerous people are involved, the centralized situation can be mitigated with a suitable decision process (like voting). [5] gives an example of using this pattern.

DP6: Push-based Inbound Oracle. The Oracles actively introduce off-chain information to the blockchain by monitoring off-chain state changes or periodically sending off-chain data to smart contracts. This approach enables on-chain components to directly access external data without time-consuming searches, while also allowing distributed Oracles to offer a dependable source of external data [2], [6], thereby improving reliability. However, because the Oracle is not triggered by the blockchain, the blockchain can not validate the data in the oracle [1] As a result, data transparency may be reduced. Additionally, off-chain applications may intentionally tamper with data, thereby avoiding on-chain validator detection and creating potential security risks. This design pattern has been implemented in the Town Crier project, proposed by Zhang et al. [46]. A smart contract from Ethereum named "Etheroll" also adopted this pattern.

Commonality from DP3 to DP6: These patterns result in decreased security, these patterns aim to address the challenge of incorporating external data into the blockchain while ensuring its trustworthiness. Since the blockchain itself cannot verify the correctness of external data, the security of the on-chain data is at risk if the external data source is tampered with or attacked. This may lead to potential security vulnerabilities. To address these shortcomings, [6] offers a contract

code implementation of this pattern in Ethereum, highlighting the contradiction with the decentralized network theorem. Meanwhile, other project[2] and study [46] attempt to improve the situation by providing data with proof of authenticity. Note that patterns of outbound Oracles (DP1, DP2) do not compromise security, as Oracles in these patterns only transmit information to external parties. And attacks on outbound data or outbound Oracle will only cause limited damage and can not affect the blockchain system directly.

Commonalities from DP1 to DP6: 1) All patterns discussed in this context aim to enhance interoperability. In spite of the fact that the purpose and method of communication with the outside world differ, they provide methods for interacting with off-chain components and improving the interoperability between on-chain and off-chain data. 2) Besides, all patterns have decreased transparency because of the dependency on off-chain components, which is also an inherent constraint. Due to the mechanism of interaction with the outside world, all of these four patterns (DP3, DP4, DP5, DP6) of inbound Oracle inevitably rely on external data, and the way off-chain data is introduced can lead to centralization and transparency risk [1], [3], [6], which directly affects the internal blockchain system. In addition, the transparency of outbound Oracle patterns (DP1, DP2) will depend on the behavior of the outbound Oracles. Using trusted third parties or decentralized deployment will increase trustworthiness of Oracle behaviors. Therefore, the transparency of these two patterns is not an inevitable tradeoff attribute and thus is not labeled in the figure.

C1.2: On/Off-Chain Association

Blockchains have distinct advantages and drawbacks. While ensuring data transparency and trustworthiness, it also presents shortages or tradeoff attributes such as high storage consumption and low computation performance [47]. Thus, it is crucial to determine which data and business logic should be stored on-chain and which off-chain to minimize system overhead. The decision to select either of these options can have varying impacts on system performance, privacy, and other quality attributes. This section gathers and analyzes various design patterns concerning the on/off-chain association and their respective quality attributes, as shown in Fig. 3.

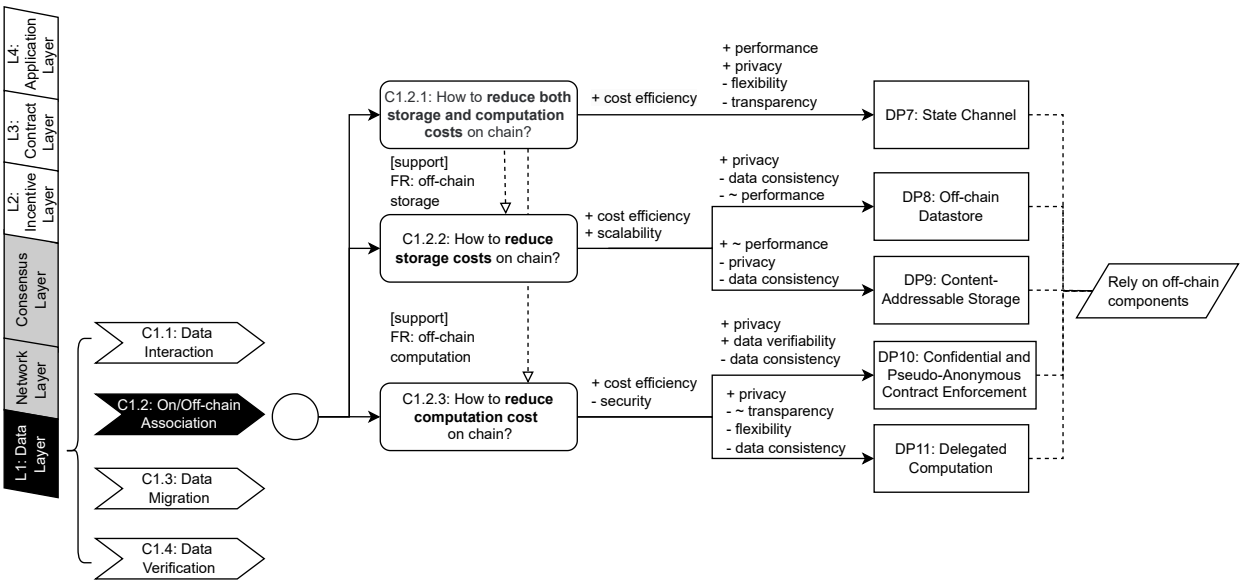

Fig. 3: Decision Model for On/Off-Chain Association (C1.2)

C1.2.1: How to reduce both storage and computation costs on chain?

DP7: State Channel. This pattern utilizes smart contracts to create off-chain payment channels for frequent and small peer-to-peer micro-transactions, with only the final outcome stored on-chain [4], [10]. Since channel transactions don't require validation by blockchain validators, channel throughput is not restricted by block configuration and consensus mechanisms, resulting in superior performance. Moreover, the intricate transaction logic of the channel is processed off-chain, with only the end result recorded on the blockchain. This approach reduces data storage and computation overhead, as well as transaction fees. Additionally, transaction privacy is ensured, as transaction details are only visible to the parties involved, and not visible to blockchain participants. This makes it highly suitable for micro-transactions [3]. However, the implementation of off-chain channels requires the introduction of additional components for extension, and the state-channel protocol will modify the existing system intrusively, making it difficult to be applied in different scenarios [4], resulting in poor flexibility. Moreover, blockchain trust in off-chain transaction protocols results in reduced data transparency, which compromises transparency [10]. There are numerous relevant examples, and one of the most classic ones is the Lightning Network[3] on the Bitcoin blockchain. It is an implementation proposal that utilizes bidirectional payment channels to enable secure payments across multiple peer-to-peer channels.

C1.2.2: How to reduce storage costs on chain?

TABLE 4: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C1.2: On/Off-Chain Association)

ID	Pattern Name	Quality Attribute	Impact	Quality Attribute Mapping Description
DP7	State Channel	Performance	Positive	Results in superior performance as channel throughput is not restricted by block configuration and consensus mechanisms.
		Privacy	Positive	Ensures transaction privacy as details are only visible to the involved parties and not to all participants.
		Flexibility	Negative	Poor flexibility due to the intrusive modification of existing systems required for off-chain channels.
		Transparency	Negative	Reduced data transparency as the blockchain relies on off-chain transaction protocols.
DP8	Off-chain Datastore	Privacy	Positive	Protects data privacy by publicly storing the hash on-chain instead of the actual information.
		Data Consistency	Negative	Difficulty in ensuring data consistency as off-chain data modification cannot be prevented or rolled back to the correct state.
		Performance	Negative	Longer transaction times due to the additional layer of hash-based indirect calls (Compare with DP9).
DP9	Content-Addressable Storage	Performance	Positive	Better querying efficiency and transaction performance due to direct references in smart contracts (Compare with DP8).
		Privacy	Negative	Compromised privacy as data references are stored publicly, allowing original data to be queried by anyone.
		Data Consistency	Negative	Cannot detect external data modification, making external storage reliability crucial.
DP10	Confidential and Pseudo-anonymous Contract Enforcement	Privacy	Positive	Enhances privacy by anonymizing user identities and encrypting sensitive information.
		Data Verifiability	Positive	This pattern includes user identity verification using smart contracts and middleware, ensuring relevant data is verifiable.
		Data Consistency	Negative	Off-chain data modification or malicious behavior cannot be prevented by contracts on-chain.
DP11	Delegated Computation	Privacy	Positive	Protects privacy by limiting the information known to third parties and keeping private data confidential
		Transparency	Negative	zero-knowledge Succinct Non-interactive ARgument of Knowledge (zk-SNARKs) is often used to prove off-chain computation, which may introduce undesirable trust in the overall system (Compare with DP10).
		Flexibility	Negative	Off-chain computation cannot be standardized, thus there is no straightforward approach to verifying data from various situations or environments.
		Data Consistency	Negative	Difficult to verify the consistency between off-chain computation data and on-chain proofs, also leading to potential security issues.
-	Commonality (DP7 to DP11)	Cost Efficiency	Positive	Reduces on-chain data storage and computation overhead, as well as transaction fees, by processing transaction off-chain.
-	Commonality (DP8 and DP9)	Scalability	Positive	Both patterns improve scalability by reducing on-chain resource consumption through off-chain storage and computation. With additional resources, off-chain computing components can increase computational capacity linearly.
-	Commonality (DP10 to DP11)	Security	Negative	Both patterns depend on off-chain components. Security concerns due to the risk of off-chain data modification or malicious behavior, as these cannot be prevented by on-chain contracts, which can indirectly affect overall security and trustworthiness.

DP8: Off-chain Datastore. The original data is recorded off-chain in this pattern, and only the corresponding hash is uploaded on-chain as a replacement. There is no need to maintain large-scale original data and blockchain overhead can be significantly reduced [3]. Hash is publicly stored on-chain instead of the actual information to protect data privacy [11]. This solution also ensures the integrity of the off-chain original data through hash-checking and can timely verify and detect data inconsistencies with the on-chain hash compared to the next pattern [12]. However, it cannot prevent off-chain data modification or roll back off-chain data to the correct state. Additionally, transactions will take longer than the following pattern due to the additional layer of hash-based indirect calls. There are many use cases such as Oraclize[4], MLGBlockchain[5] and Hawk [48].

DP9: Content-Addressable Storage. The data is kept off-chain in this pattern, while the Smart Contract (SC) records data references, or data addresses [10]. This allows original data to be stored off-chain, reducing blockchain storage costs. Due to the fact that SC records are directly referenced, this pattern provides better querying efficiency and better transaction performance, and can be used to share user on-chain information more easily than Off-chain Datastores. However, privacy is compromised because references are stored publicly, and the original data can be queried by anyone. Therefore, access control mechanisms can be introduced to restrict visitors. Moreover, this pattern cannot detect external data modification, unlike the other patterns, so external storage must be reliable. Interplanetary File System (IPFS) [49] and Swarm [50] are two advanced experimental projects with content-addressable storage.

Commonalities for DP8 and DP9: 1)Both patterns have proven effective in reducing system costs and improving scalability. Off-chain Datastore (DP8) hashes data on-chain while Content-Addressable Storage (DP9) stores data references.

Both methods significantly decrease on-chain data storage and processing, resulting in reduced storage and computational costs [3], [4]. Transaction fees can also be reduced as a result [11]. Particularly in large systems, they can achieve the desired increase in storage capacity by adding hardware and software resources to the off-chain database, effectively coping with data expansion and maintaining better scalability. 2)Both of these patterns reduce data access performance, since on-chain data is either hashes or references. As a result, data reading and writing require off-chain database access instead of direct access to the blockchain, leading to a decrease in data access performance. Furthermore, these methods introduce extra communication mechanisms and platforms for data interoperability. This can complicate data sharing on-chain and hinder collaboration among blockchain participants [3]. 3)Both patterns pose difficulty with data consistency. While using data hash or reference reduces on-chain costs, it does not prevent off-chain data modification or loss, nor facilitates rollback of data to its correct state [3], [4], [12]. Hence, maintaining the correct mapping between on-chain abstract data and off-chain original data becomes difficult. Additionally, the Content-Addressable Storage pattern cannot detect data tampering, unlike the other one that can verify data correctness using hash-checking. This tradeoff attribute may also have an indirect negative impact on security.

C1.2.3: How to reduce computation costs on chain?

DP10: Confidential and Pseudo-anonymous Contract Enforcement. Here, the execution state is stored in the on-chain smart contract, and the logic to execute it is wrapped off-chain, reducing on-chain computational overhead. The publicly stored execution state can serve as proof or evidence in dispute resolutions. This pattern includes user identity verification using smart contracts and middleware, ensuring relevant data is verifiable [18]. Additionally, privacy can be protected because user identities are anonymized and inherently sensitive information about users is encrypted. However, as mentioned above, off-chain comments may become a target of attack and be injected with false data [51]. Off-chain data modification or malicious behavior cannot be prevented by contracts on-chain. Thus, the data consistency and security of this pattern needs to be improved. And the case of transportation of refrigerated goods is illustrated in [18].

DP11: Delegated Computation. This pattern outsources computation to untrusted third-parties or clients, and state checks of the process can be conducted by proof generation and verification, leading to reduced computation costs and enhanced privacy protection [10]. Only the necessary information related to the calculation is known to the delegated third-parties, and the private data remains confidential. Moreover, this pattern offers enhanced security over the previous one, as malicious off-chain computation behavior cannot generate the correct proofs needed for on-chain validation, under ideal conditions. Proof complexity can be independent of off-chain computation complexity by design. Thus, after a certain threshold, on-chain verification is faster than on-chain computation, leading to better throughput and lower overhead, especially for complicated computation tasks. However, zero-knowledge Succinct Non-interactive ARgument of Knowledge (zkSNARKs) is often used to prove off-chain computation, which may introduce undesirable trust in the overall system [52], [53], causing more decrease in transparency than other patterns. Moreover, off-chain computation cannot be standardized, and there is no straightforward approach to verifying data from various situations or environments, leading to limited applicability and flexibility [10]. In addition, off-chain computation data may not match on-chain proofs. This pattern is often applied in specific cases that implement privacy preserving transactions, such as ZCash[6], which is proposed in [54].

Commonalities for DP10 and DP11: 1) Both reduce system costs and improve scalability. While both patterns aim to protect privacy, they also reduce on-chain computing overhead. This is achieved by transferring computing tasks to off-chain components, simplifying the on-chain computation process. One pattern records computational states on-chain [18], while the other verifies proofs on-chain [10]. With additional resources, off-chain computing components can increase computational capacity linearly, leading to better scalability for compute-intensive tasks that are rapidly expanding. 2) Both patterns improve privacy. Both of these patterns enhance privacy protection. The first pattern (DP10) ensures privacy through user pseudonymity and process confidentiality, while the second pattern (DP11) achieves privacy protection by hiding unnecessary and private information. And zero-knowledge can be used in case the prover leaks privacy. [10] 3)Both patterns suffer from data consistency and security issues. As previously noted, since only states or proofs are stored on-chain, the complete transaction process and detailed data cannot be recorded on the blockchain. As a result, establishing a complete mapping between on-chain data and off-chain data becomes challenging, leading to poor data consistency [10], [18]. And this inconsistency will decrease security due to the fact that each off-chain data carries the risk of being tampered with without verification by the blockchain.

Commonality from DP7 to DP11: These five patterns all depend on off-chain components, as an inherent attribute. This feature may have different impacts on attributes such as data transparency, traceability, and consistency. Although we didn't find a direct discussion about security, these tradeoff attributes can indirectly impact the overall security and trustworthiness of the system. And in general, the potential risks to blockchain posed by external data need to be considered from multiple perspectives during interaction.

C1.3: Data Migration

The immutability of blockchain can be a drawback in certain scenarios due to rapid changes in system requirements. Once a blockchain system is deployed, making any changes, such as replacing the platform, adjusting the business model, or migrating data, can be more difficult compared to traditional software. To keep blockchain applications competitive and secure, it is necessary to summarize migration patterns [19]. This section analyzes these patterns and discusses the relevant quality attributes, as shown in Fig. 4.

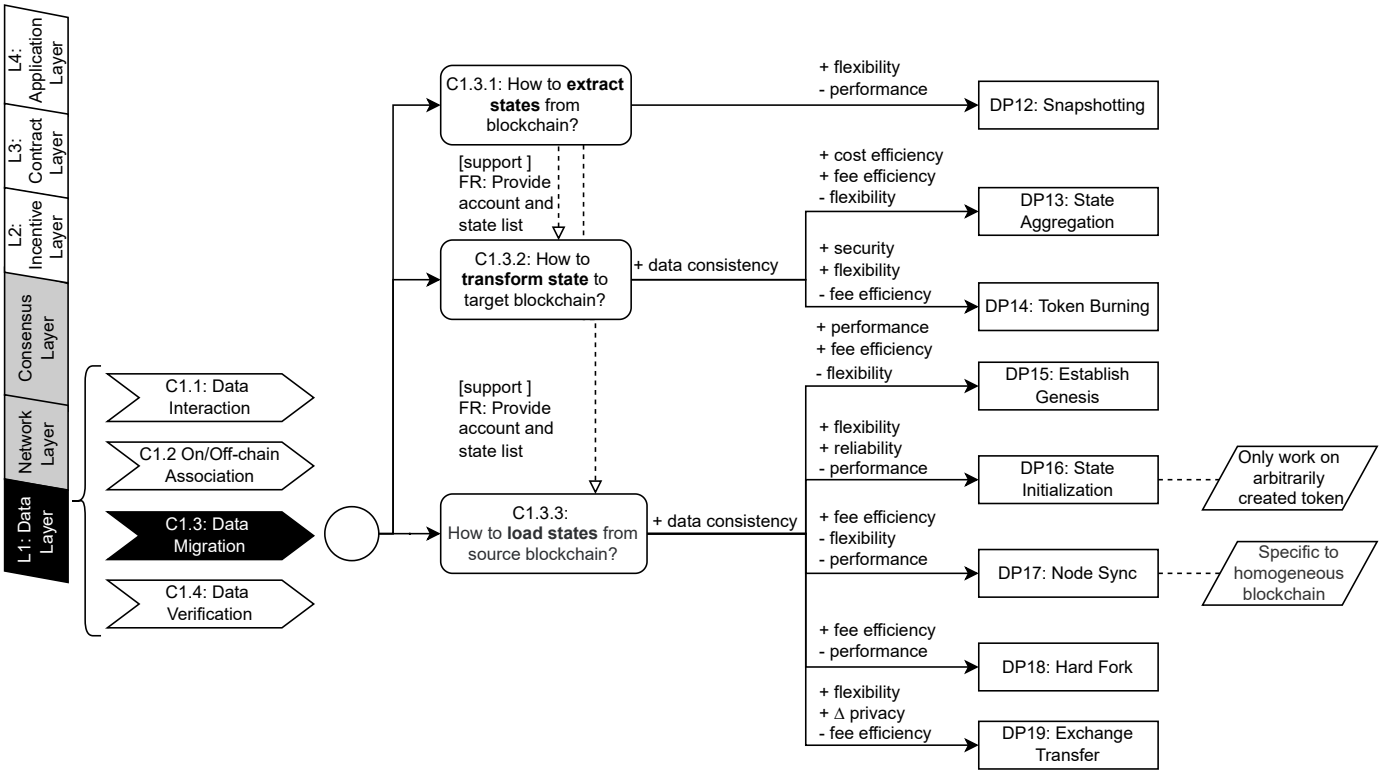

Fig. 4: Decision Model for Data Migration (C1.3)

C1.3.1: How to extract states from blockchain?

DP12: Snapshotting. This is a method of accessing blockchain information, including transactions, account statuses, and smart contracts, in order to generate a globally consistent view of the blockchain at a specific point in time [11]. This pattern involves creating a snapshot after reaching finality to maintain consistency among different blockchain nodes for data migration. The flexibility of this pattern allows for the generation of snapshots of any combination of smart contracts, statuses, and transactions. It can create snapshots of any blockchain. However, it should be noted that this pattern may result in increased latency as the time required to generate snapshots is proportional to the number of accounts and smart contracts, including their status and transactions. Furthermore, latency may also be impacted by the time blockchain to reach finality. Therefore, this pattern is most suitable for blockchains without external interaction [19]. There are many use cases such as TOMO[7], Tron[8], and Vechain[9], etc.

C1.3.2: How to transform states to target blockchain?

DP13: State Aggregation. This pattern consolidates a group of states into a single state (e.g., combining account assets into a single account) [19]. Aggregation/disaggregation operations ensure better data consistency. The Snapshotting pattern can also be used to provide state lists for aggregation. After aggregation, the storage and computational costs of accounts and states can be significantly reduced during migration. In public blockchains, this pattern can lower transaction fees, provided that the number and complexity of accounts are not too high. However, this pattern can only be applied to migrations of simple states, which means the aggregation process can be simplified as the merging of several values. This limitation makes the pattern unsuitable for smart contracts with complex states and reduces its flexibility. In practical use cases, the pattern has been applied during the bootstrapping of tokens on the target blockchain, such as Storj[10] and Binance[11].

DP14: Token Burning. In order to get immutable Proof of Burn (PoB) and remove redundant assets, the tokens and states of the source chain are set to unavailable when migrating. Because of PoB's immutability and accountability, tokens or states on the source blockchain can be destroyed for verification on all target blockchains for better migration consistency [11]. This pattern can prevent the misuse of states and smart contracts by burning, thus avoiding double-spending attacks and improving security. Besides, PoB can be used in all blockchains, resulting in a broader application range and better flexibility when data migration. However, burning tokens or states for each transaction incurs additional overhead compared to normal transactions, resulting in higher transaction fees. Some of the cases of the Token Burning pattern in practice include Bitizens[12], KARMA[13], and Kin[14], which burned tokens during their migration from Ethereum or Bitcoin to their own blockchain instances.

C1.3.3: How to load states from source blockchain?

DP15: Establish Genesis. This pattern facilitates the migration of states to the genesis block of the targeted blockchain using snapshots [19]. Proof of Existence (PoE) enables users to verify the existence of data at a specific time by storing its

TABLE 5: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C1.3: Data Migration)

ID	Pattern Name	Quality Attribute	Impact	Quality Attribute Mapping Description
DP12	Snapshotting	Flexibility	Positive	By allowing for the generation of snapshots at any combination of smart contracts, statuses, and transactions, on-chain data migration can be flexible.
		Performance	Negative	May result in increased latency as the time required to generate snapshots is proportional to the number of accounts and smart contracts.
DP13	State Aggregation	Cost Efficiency	Positive	Reduces storage and computational costs by aggregating multiple states into a single state, simplifying state management during migration.
		Fee Efficiency	Positive	In public blockchains, this pattern can lower transaction fees, provided that the number and complexity of accounts are not too high.
		Flexibility	Negative	Poor flexibility due to the limitation of being applicable only to migrations of simple states.
DP14	Token Burning	Security	Positive	Enhances security by preventing misuse of states and smart contracts through Proof of Burn (PoB).
		Flexibility	Positive	PoB can be used in all blockchains, resulting in a broader application range and better flexibility.
		Fee Efficiency	Negative	Burning tokens or states for each transaction incurs additional overhead compared to normal transactions.
DP15	Establish Genesis	Performance	Positive	Increases performance by quickly recreating states in the genesis block without requiring transaction fees.
		Fee Efficiency	Positive	The absence of transaction fees is an advantage, since only the genesis block is involved.
		Flexibility	Negative	Poor flexibility due to the constraint that the states must fit into a single block, and creating new accounts will slow down the process.
DP16	State Initialization	Flexibility	Positive	Offers better flexibility by allowing state initialization on any target blockchain, irrespective of its governance approval.
		Reliability	Positive	Improves reliability by handling each state individually, reducing the risk of human-caused errors.
		Performance	Negative	It requires the creation of individual accounts for each on-chain state, leading to increased latency and cost.
DP17	Node Sync	Fee Efficiency	Positive	It synchronize nodes at the data structure level without requiring transaction fees.
		Flexibility	Negative	Poor flexibility as it is specific to only homogeneous blockchains.
		Performance	Negative	Require considerable time for blockchain system to synchronize large amounts of state and transaction history.
DP18	Hard Fork	Fee Efficiency	Positive	Eliminates transaction fees by making state changes in the global state without requiring transaction-based operations.
		Performance	Negative	Reduced performance due to the time required for multiple nodes to synchronize after a hard fork.
DP19	Exchange Transfer	Flexibility	Positive	Enhances flexibility by supporting the migration of states through cryptocurrency/token exchanges without requiring blockchain governance approval.
		Privacy	Positive	Improves privacy by using decentralized exchanges for token transfers, making the process more private compared to centralized solutions.
		Fee Efficiency	Negative	Potentially increases transaction fees depending on the data volume, although batch or centralized exchanges can reduce costs.
-	Commonality (DP13 to DP19)	Data Consistency	Positive	All patterns maintain data consistency during migration by preserving the mapping between source and target blockchain states.

digital fingerprint (hash) on the blockchain [55]. The PoE entry is integrated with the snapshot to ensure consistent state mapping before and after data migration. Because this pattern only uses genesis blocks, the states can be quickly recreated, and throughput can be improved. Additionally, the absence of transaction fees is an advantage of this approach, since only the genesis block is involved in the migration process. However, it works only if the states can be fit into a single block, and creating new accounts will slow down the migration process [19]. This constraint reduces the pattern's flexibility and applicability to various scenarios. Projects like Telos[15] and Bithereum[16] have utilized snapshot files to construct the genesis block during the migration process.

DP16: State Initialization. This pattern initializes a new account on the targeted blockchain for state rebuild, which is suitable for any target blockchain regardless of its existence or blockchain governance approval [19], and achieves better flexibility. Similarly, PoE entry is also introduced for data consistency. In addition, this pattern can handle only one state or smart contract each time, reducing the risk of human-caused errors and providing better reliability. However, the downside of this pattern is that it requires the creation of individual accounts for each state, leading to increased latency and cost. This constraint can be alleviated by adopting the State Aggregation pattern. Moreover, as a constraint, this pattern only works on arbitrarily created tokens, like the tokens generated and maintained by smart contracts. Native tokens like BTC or ETH are not suitable. This pattern has been utilized by numerous projects, such as Gifto[17], Kin [18], and Qubicle[19], to issue new tokens on the target blockchain.

DP17: Node Sync. This pattern realizes node data cloning by synchronizing data such as smart contracts, transactions,

and states of source chain nodes. This pattern ensures state and log consistency during data transfer by using asynchronous messaging and consensus mechanisms [19]. In addition, no transaction fee is required in this pattern because the data is transferred at the data structure level. However, this model requires both sides to be homogeneous blockchains, which is an inherent constraint. Hence, it cannot be adapted to all situations, which causes less flexibility. Also, nodes can take long to synchronize large amounts of state and transaction history, resulting in low performance. This problem can be alleviated by the Snapshotting pattern for faster synchronization speed, and the blockchain governance body's approval is also not required [19]. This pattern is used to connect new nodes to an existing blockchain, such as Bithereum [23]. Ethereum also supports syncing data at different levels, such as full, fast, and light [56].

DP18: Hard Fork. This pattern changes the global state of the target blockchain using snapshots for data migration, and the state changes can violate the consensus rules. Therefore, it causes a hard fork on the target blockchain, which changes blockchain protocol radically and maintains a different version of block data in a small group of nodes [19]. Because the blockchain global state is changed by the hard fork, there is also no transaction fee here. In the same way, PoE entries can also be used to track the mapping between old and new records for better accountability and consistency. However, due to the initiation of a new fork involving consensus of multiple nodes, the time required to synchronize can be long. In comparison to Node Sync, this pattern requires the governing body's agreement to initiate a hard fork, leading to more waiting time and lower performance. Private and consortium blockchains are more suitable because of the requirement of blockchain government body consensus. This pattern can be used to migrate or spin-up cryptocurrencies, like Æternity[20], Steem[21].

DP19: Exchange Transfer. This pattern migrates the states through the cryptocurrency/token exchange [19]. Each transaction on token exchange and state transfer during migration is validated and recorded on-chain for consistency. This pattern supports both blockchain native assets and tradeable tokens, and is suitable for various blockchains without blockchain governance approvals, leading to better flexibility. Additionally, the token exchange's disintermediation and decentralization make it more private compared to centralized solutions [19]. However, since costs are related to data volume, this method may result in more transaction fees. But the cost can be reduced through batch or centralized exchange. And in practice, EOS[22] and Interledger[23] provide protocols to transfer tokens via decentralized exchanges. Many other projects require users to transfer their assets to a designated exchange.

Commonality from DP13 to DP19: Each of them preserves data consistency during data migration. Data consistency refers to the completeness of mapping between associated data on the source and target blockchains, which is essential for successful data migration. The State Aggregation pattern (DP13) uses aggregation/disaggregation operations to provide better data consistency before and after migration. The Token Burning pattern (DP14) also preserves consistency through proof of burn (PoB), which requires the burning of equivalent assets from the source blockchain to recreate account data or states on the target blockchain. Establish Genesis (DP15), State Initialization (DP16) and Hard Fork (DP18) all integrate PoE to ensure data consistency. Node Sync (DP17) takes advantage of asynchronous messaging and consensus mechanisms. Finally, the Exchange Transfer (DP19) pattern records the migration process on the blockchain to achieve better consistency.

C1.4: Data Verification

Signature and verification are fundamental concepts in blockchain technology. However, the practical applications of these concepts are complex, and effectively protecting user privacy can be challenging. Situations such as key loss, malicious attacks, and illegal access pose serious challenges to data security [57]. There is still a need for reasonable design patterns to ensure signature verification and data privacy are secure, trustworthy and reliable. It is worth noting that some patterns in this section focus on contract data security, which is distinct from the contract security patterns (C3.4) that concern contract encoding and execution security rather than data. Moreover, although the network layer contains data verification, it refers to the validation of the received block's validity by validators. In contrast, this section concerns user account and key management, which focus on transaction data and privacy data of users. Essentially, these patterns are still management of on-chain data rather than blockchain networks. Consequently, in this data layer, we consider the patterns to be designed to protect the confidentiality of personal privacy. Overall, this section includes patterns concentrating on data security and privacy, including secret key management and encryption methods, as shown in Fig. 5.

C1.4.1: How to manage on-chain keys?

DP20: Master and Sub Key Generation. In this pattern, each user can manage the subkeys used in different identities with a master key [11]. For instance, an individual may use distinct subkeys for their student and citizenship identities when conducting transactions. It is difficult to track and analyse associations of these identities, and personal privacy can be protected. The subkey can be regenerated by the master key after it is lost, and thus the system can timely recover from a failure, leading to better availability. However, if the master key is compromised, control over all associated subkeys will be lost, potentially leading to disastrous consequences. Consequently, this pattern may be considered less secure than others, and users must exercise increased vigilance in safeguarding their master keys. Some use cases of this pattern include uPort[24] and Trinity[25], where the master key is utilized to control specific accounts.

DP21: Hot and Cold Wallet Storage.In this pattern, keys are kept in two types of wallets: hot and cold wallets. Through hot wallets, users can easily operate their frequently-used accounts with keys, while keeping their infrequently-used keys offline through cold wallets. In the event that the hot wallet becomes compromised by malicious attacks, the keys stored offline in the cold wallet can be directly accessed after connecting to the network, thereby allowing the system to recover from abnormal status and provide better availability and security. However, compared to other patterns, the security in this

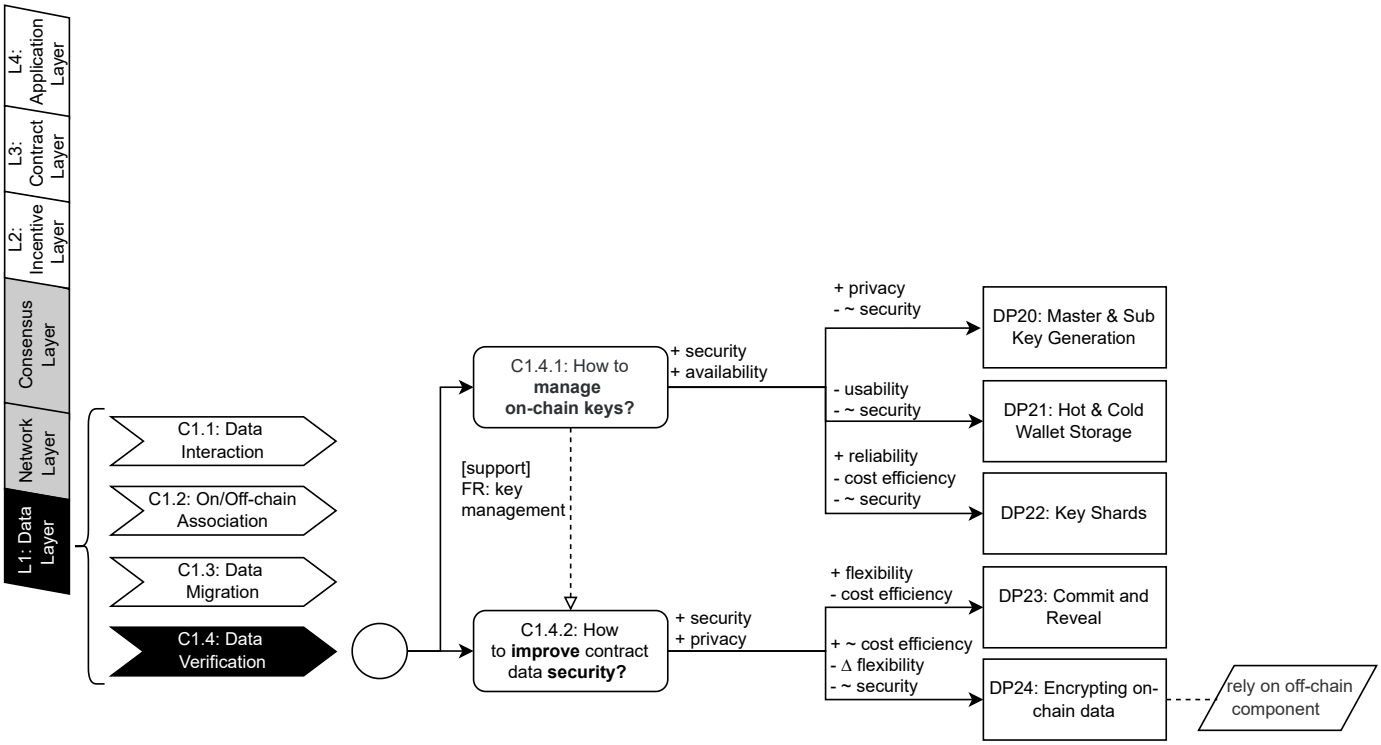

Fig. 5: Decision Model for Data Verification (C1.4)

TABLE 6: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C1.4: Data Verification)

ID	Pattern Name	Quality Attribute	Impact	Quality Attribute Mapping Description
DP20	Master and Sub Key Generation	Privacy	Positive	Enhances privacy by allowing users to manage subkeys for different identities, making it difficult to track and analyze identity associations.
		Security	Negative	As compromising the master key can lead to the loss of control over all associated subkeys, security is compromised (Compare with DP21, DP22).
DP21	Hot & Cold Wallet Storage	Usability	Positive	Provides better usability by enabling users to easily operate frequently-used accounts with hot wallets while keeping infrequently-used keys offline in cold wallets.
		Security	Negative	The hot wallet, while storing sensitive keys offline, remains vulnerable to theft, which partially compromises security (Compare with DP20, DP22).
DP22	Key Shards	Reliability	Positive	Improves reliability by ensuring system availability as long as the threshold of key shards is met, even if some shards are lost or damaged.
		Cost Efficiency	Negative	maintaining key shards requires additional resources and overhead.
		Security	Negative	storing the key shards remain critical targets for attack and the damage to the shards themselves cannot be recovered, leading to lower security (Compare with DP20, DP21).
DP23	Commit and Reveal	Flexibility	Positive	Enhances flexibility by enabling the intended execution order of smart contracts through concealed value commitments.
		Cost Efficiency	Negative	an additional on-chain mechanism is required to hide contract variables, leading to increased on-chain computation.
DP24	Encrypting On-chain Data	Cost Efficiency	Positive	Encryption on-chain data is a more efficient and cost-saving privacy mechanism (Compare with DP23).
		Flexibility	Negative	unlike traditional systems, revocation of participant access to specific data is not possible since once on-chain data is accessed, it can be accessed indefinitely.
		Security	Negative	While the encryption mechanism objectively enhances security, dependency on off-chain components can lead to compromise and disclosure of private keys.
-	Commonality (DP20 to DP22)	Security	Positive	All patterns improve security by providing various approaches to protect secret keys. The malicious behaviors focused on secret keys can only have a limited impact.
		Availability	Positive	Ensures high system availability by offering effective strategies to handle compromised keys and recover quickly.
-	Commonality (DP23 to DP24)	Security & Privacy	Positive	Both patterns enhance security and privacy protection through encryption and concealed value commitments, ensuring that sensitive information is protected.

pattern is still compromised as the keys stored in hot wallets are still vulnerable to theft. Additionally, using cold wallets may be inconvenient for users as the cold wallet device may not be ready to use, leading to poor usability [11]. This pattern does not result in off-chain component dependency, as the cold wallet does not participate in on-chain operations while offline. Trezor[26] and Ledger[27] are hardware wallets designed to securely store users' private keys for cryptocurrencies.

DP22: Key Shards.This pattern splits the key into multiple shards, which can only be used when the user has a sufficient number of shards [58]. This approach enables the key to be stored in pieces, reducing the impact of shard loss or damage on the storage of the key itself. The system remains available as long as the threshold of key shards is met, and can quickly recover from exceptions such as key shard loss or leak, leading to improved reliability and availability. Additionally, this pattern can also mitigate external attacks. However, the nodes storing the key shards remain critical targets for attack and the damage to the shards themselves cannot be recovered [11]. Moreover, maintaining key shards requires additional resources and overhead. Parity[28] and Crypto++[29] provide key recovery mechanisms based on shards.

Commonality from DP20 to DP22: All of the patterns provide solutions for better security while maintaining high availability. These strategies incorporate a variety of approaches to protect the secret key, such as utilizing master and sub keys, online and offline storage, and key shards. The malicious behaviors focused on secret keys can only have a limited impact. Furthermore, the system is equipped to promptly recover from any potential breaches because all of the patterns offer effective strategies to handle compromised keys and swiftly transition to well-secured keys, thus ensuring high system availability.

C1.4.2: How to improve contract data security?

DP23: Commit and Reveal. This pattern conceals and commits a secret value without knowledge of other entities' committed values in the smart contract. This approach ensures enhanced privacy and security, with values or parameters being revealed only after transaction confirmation at a specific point in time [6]. The pattern treats all committed values equally, and their order is independent of the transaction submission order. This feature enables the intended execution order of smart contracts to be realized, resulting in improved flexibility [24]. This pattern is versatile and can be applied to many scenarios, including voting schemes and lotteries. However, an additional on-chain mechanism is required to hide contract variables, leading to increased on-chain computation and system costs [18]. Consequently, transaction fees will also increase. A Solidity example is provided in [6].

DP24: Encrypting On-chain Data. This pattern utilizes encryption to secure critical on-chain data, which can only be accessed by authorized participants who hold the key managed off-chain [15], [16]. This approach provides a more efficient and cost-saving privacy mechanism than the Commit and Reveal pattern. However, unlike traditional systems, revocation of participant access to specific data is not possible since once on-chain data is accessed, it can be accessed indefinitely in the future, causing less flexibility. Additionally, the private key can only be shared off-chain due to the public accessibility of information communicated through the blockchain [17], which poses potential security risks and indirect impacts on transparency and privacy. While the encryption mechanism objectively enhances security, dependency on off-chain components can lead to compromise and disclosure of private keys [3], further highlighting the importance of carefully considering the trade-offs between security and other factors. Some projects such as Oraclize, MLGBlockchain, and Hawk [48] offer encryption for data, including queries and transactions.

Commonality for DP23 and DP24: Both patterns provide solutions to improve security and effectively protect privacy. Commit and Reveal hides parameters in the smart contract while Encrypting On-chain Data secures data through on-chain encryption with secret keys. Through the use of encryption, sensitive information and privacy data can be protected and avoid attacks like injection attacks [4], making it difficult for unauthorized participants to access.

L2: Incentive Layer

Blockchain deterministic assurance relies on an honest majority, so incentive mechanisms are used to ensure it [59]. This layer pertains to economic considerations in the blockchain ecosystem, which typically involves the block mining reward system and token transactions. The issuance mechanism and distribution mechanism are included in this layer [12]. It is mainly implemented on public blockchains with tokens like BTC or ETH. This layer focuses on the incentive mechanism including token issuance.

C2.1: Issuance Mechanism

The issuance mechanism primarily refers to the reward system for miners. But with the growing popularity of blockchain payments and the increasing representation of real-world assets on-chain, the issuance mechanism has evolved. Tokens on-chain can also represent real-world assets. The transfer of ownership of assets is replaced by token transactions on-chain, reducing transaction risks and increasing efficiency. In this section, we will discuss the design pattern related to token issuance, known as tokenisation, as shown in Fig. 6.

C2.1.1: How to value on-chain assets via token?

DP25: Tokenisation. This pattern defines circulating currencies/tokens among participants, representing transferable digital or physical assets, and facilitating the flow of tokens based on smart contracts [8]. Transactions or trades in the real-world can be decentralized and recorded by blockchain. Inherently, this pattern optimizes traditional finance rather than blockchain itself. Therefore, most quality attributes discussed here are compared with traditional finance. With blockchain, tokens can allow for better asset ownership tracking, regulating user authorization and identity, and better guaranteeing

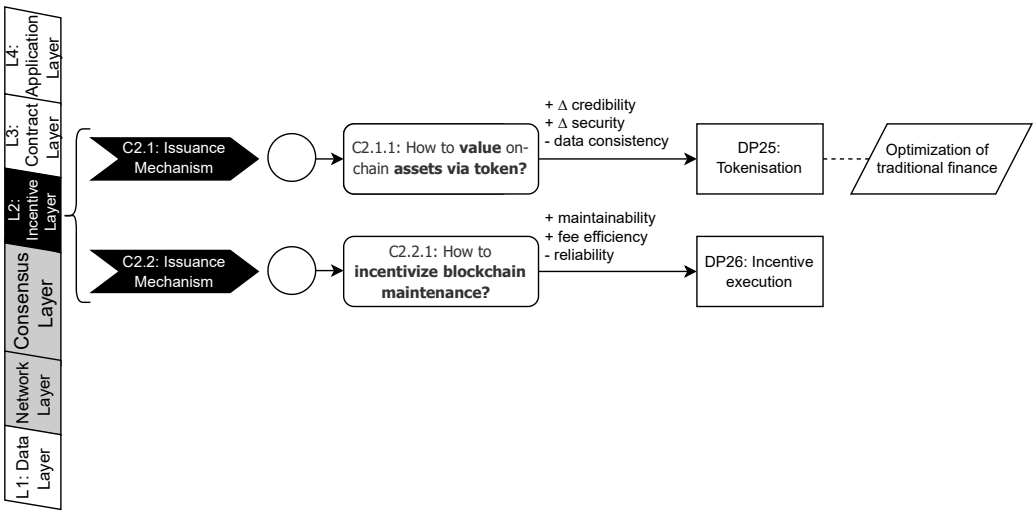

Fig. 6: Decision Model for Issuance Mechanism (C2.1)

TABLE 7: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C2.1 & C2.2: Issuance and Distribution Mechanism)

ID	Pattern Name	Quality Attribute	Impact	Quality Attribute Mapping Description
DP25	Tokenisation	Credibility	Positive	Facilitates better tracking and regulation of asset ownership and identity, enhancing credibility and transparency in asset circulation compared to traditional finance.
		Security	Positive	Reduces the risk of handling high-value financial products, ensuring more secure transactions than traditional methods.
		Consistency	Negative	On-chain tokenization can only guarantee the security and trustworthiness of on-chain tokens, but not the real-world product itself.
DP26	Incentive Execution	Maintainability	Positive	Improves maintainability by incentivizing users to perform maintenance tasks through economic rewards.
		Fee Efficiency	Positive	Reduces transaction fees by providing token rewards for executing utility contracts.
		Reliability	Negative	The user-initiated nature of utility contracts poses a risk of unreliable execution, potentially leaving the system unmaintained for extended periods.

asset circulation transparency [60]. Tokenization also reduces the risk in handling high-value financial products, ensuring a more secure transaction than traditional methods [3]. However, similar to patterns with off-chain components, on-chain tokenization can only guarantee the security and trustworthiness of on-chain tokens, but not the real-world product itself. This lack of consistency between on-chain tokens and off-chain products can be a drawback [5]. Other drawbacks may also be restriction, such as the threat of illegal ownership processes [3]. There are many practical cases, such as the DVIP smart contract on ETH for token rights [8], the definition of token interface in ERC20[30], and the Digix[31] for tracking gold ownership.

C2.2: Distribution Mechanism

The distribution mechanism focuses on the token allocation. This mechanism restricts token circulation by evaluating each node's contribution and incorporating random factors to ensure equity and impartiality [61]. But under certain circumstances, the native allocation mechanism may be unreasonable. It is necessary to modify the mechanism to incentivize other behaviors that contribute towards better maintenance of the overall blockchain system. In this section, we will discuss the design pattern related to token distribution, which is the incentive execution pattern, as shown in Fig. 6.

C2.2.1: How to incentivize blockchain maintenance?

DP26: Incentive Execution incentivizes users to invoke the utility contract through token rewards [3]. These contracts perform tasks such as clearing expired records, deleting obsolete contracts, and making dividend payouts. By means of economic reward, utility contracts can reduce the effort required to maintain the system and resolve issues within the blockchain, and can achieve better maintainability. This mechanism can also reduce transaction fees due to the reward. However, since utility contracts are user-initiated, there is a risk of unreliable execution that may result in extended periods of system unmaintained. Furthermore, it is worth noting that the integration of these functions should not be tied to asset-sensitive or frequently-used functions [4]. Regis[32] and Ethereum Alarm Clock[33] offer incentives to encourage users to execute scheduled functions.

L3: Contract Layer

The contract layer is mainly related to smart contracts. This layer encapsulates the on-chain script and algorithm, which is the foundation for business logic implementation in the blockchain. By using the contract layer, Turing completeness and flexibility can be promised. The use of advanced programming languages has made smart contracts similar to traditional programming and has brought increased attention to maintainability, including the ability to extend, modify, and test contract functionality. In this layer, we gather design patterns for smart contracts, including authorization, cost optimization, efficiency, security, and migration.

C3.1: Contract Authorization

Compared to traditional software development, in the decentralized environment, smart contracts face more challenges in terms of security, reliability and complexity. These challenges include the execution of critical operations or ownership transfer. Therefore, additional methods need to be designed to ensure permission control and authorization. And the patterns of this concern are mainly focused on the authorization solution of smart contracts, as shown in Fig. 7.

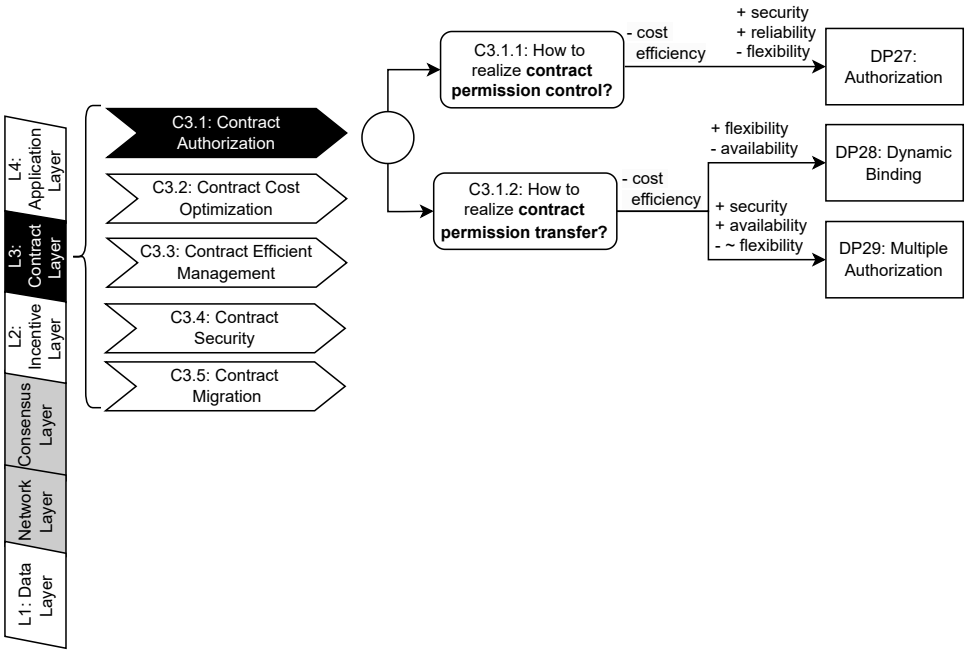

Fig. 7: Decision Model for Contract Authorization (C3.1)

TABLE 8: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C3.1: Contract Authorization)

ID	Pattern Name	Quality Attribute	Impact	Quality Attribute Mapping Description
DP27	Authorization	Security	Positive	Ensures that only authorized entities can invoke the contract, reducing the risk of malicious calls.
		Reliability	Positive	Restricts and regulates authorized operations according to owners' needs, enhancing reliability.
		Flexibility	Negative	Low flexibility due to predefined permissions and the need to determine prerequisites before deployment.
DP28	Dynamic Binding	Flexibility	Positive	Allows dynamic association of contracts with authorized user addresses, improving adaptability to changing environments.
		Availability	Negative	Compromised availability if the secret key is lost, as there is no reliable way to revoke it.
DP29	Multiple Authorization	Security	Positive	Enhances security by requiring multiple signatures for transaction validation, making it difficult for malicious behavior to affect authorization.
		Availability	Positive	Improves availability by allowing transactions to proceed even if some keys or signatures are compromised.
		Flexibility	Negative	Reduced flexibility due to predefined authorizers and the need for all authorized addresses to be determined beforehand.
-	Commonality (DP27 to DP29)	Cost Efficiency	Negative	All three patterns introduce additional overhead due to access control mechanisms, resulting in increased computation, storage costs, and potentially higher transaction fees.

C3.1.1: How to realize contract permission control?

DP27: Authorization. In this pattern, functions are executed only if certain prerequisites are met. These prerequisites may include verifying information such as the caller's permission, invoke time, and transaction details, before the contract is called [8], [22]. This ensures that only authorized entities can invoke the contract, thereby reducing the risk of malicious calls and achieving better security. Furthermore, authorized operations can be restricted and regulated according to the owners' needs [8], resulting in a higher level of reliability. However, the methods of prerequisites checking need to be determined before contract deployment. And permissions of callers are also basically predefined in the smart contract [4], so it is difficult to cope with dynamic changes of permission, resulting in low flexibility. In addition, this pattern requires extra implementation of permission control mechanisms that can bring more overhead of storage and computation in terms of contract deployment and execution. Especially on a public blockchain, this pattern may cause extra transaction fees [3]. There are several examples of Ethereum contracts that restrict code execution, such as the Corporation[34] and CharlyLifeLog[35] contracts. Also, projects like OpenZeppelin[36] introduced specific owners of contracts.

C3.1.2: How to realize contract permission transfer?

DP28: Dynamic Binding. In this pattern, contracts and its authorized users' addresses are dynamically associated. The authorization of the contract can be forwarded to unknown authorities through a secret key sent by the user [16]. This flexible and collaborative permission control method allows for better adaptability to changing environments [21]. However, availability may be compromised if the secret key is lost as there is no dependable way to revoke it. This can also hinder the system from restoring normal operation and cause indirect security risks. Additionally, the increase in functionalities and communication could result in more blockchain costs. In practice, projects such as Raiden Network[37] and Atomic Cross-Chain Trading[38] utilize off-chain hash secrets to facilitate value transfer.

DP29: Multiple Authorization. This pattern is similar to multiple signatures. Transactions are only valid when there are enough signatures. As a result, dynamic blockchain authentication can be achieved, which helps with participant collaboration [16], [21]. The Multiple Authorization pattern allows for the compromise of keys or signatures, resulting in better availability. A subset of predetermined addresses is authorized to execute transactions in this pattern. For example, an M-of-N multi-signature means that M out of N private keys are necessary for transaction authorization, where N is predefined. Therefore, as long as most private keys are kept securely, malicious behavior cannot affect authorization, resulting in better security. However, while this pattern also provides a dynamic way to collaborate, addresses that can authorise a transaction are required to be predefined [21]. Hence, flexibility is reduced because different scenarios may necessitate diverse assignments of authorizers, which are different from Dynamic Binding using unknown destined addresses directly. Moreover, the addresses of authorization and their relationships need to be realized and deployed, causing more storage and computation overheads [15]. This classical approach has been adopted in Bitcoin[39], and other multi-signature wallets such as Mist[40].

Commonality from DP27 to DP29: All three patterns introduce additional overhead to the system. Access control mechanisms require extra steps such as permission verification, authorization binding, and authorizer maintenance. These mechanisms lead to increased computation and storage costs due to the added functionality and contract calls, which can also result in higher transaction fees.

C3.2: Contract Cost Optimization

The execution of smart contracts on the blockchain usually comes with high costs in terms of storage and computation. The use of resources can also lead to an increase in transaction fees. Hence, enhancing contract execution efficiency and optimizing resource consumption are necessary. Different from these storage-saving patterns in data layer (C1.2), though there are certain shared features, the patterns here focus on optimizing smart contract code instead of modifying the underlying mechanism at the data layer. This section discusses existing design patterns for smart contracts for saving overhead and reducing transaction fees, as shown in Fig. 8.

C3.2.1: How to reduce contract costs of blockchain?

DP30: Tight Variable Packing. This pattern is employed to maintain compactness of constant strings and minimize on-chain data by storing static variables in the smallest possible data type that accommodates them [14]. As a result, on-chain costs can be reduced. This pattern typically lacks tradeoff attributes as it is comparable to traditional programming styles that aim to minimize costs. But, its positive impact is amplified when applied to blockchain-based software [4].

DP31: Limit Storage. This pattern requires for minimizing read and write operations with the blockchain database during contract design to reduce overhead. This can be achieved by saving intermediate contract data in memory or stack, and only permanently storing relevant data in the blockchain at the end of all computations [14]. As a result, interaction with the blockchain database is minimized, which leads to improved performance. Although this pattern shares similarities with traditional programming practices, the positive impact of reducing on-chain costs in blockchain-based software is significant, and the tradeoff point is not immediately apparent, as it is also not a best practice to frequently invoke the database in traditional programming. And according to Marchesi et al. [14], these two patterns are discussed through sample code in coinmonks[41].

Commonality for DP30 and DP31: Both of these patterns reduce on-chain overhead in smart contracts. When designing a contract, it is supposed to minimize the size of variables or limit data operations on-chain in order to avoid excessive resource usage. These patterns are recommended because they are similar to traditional programming practices without too many drawbacks. Transaction fees can also be saved accordingly in most cases.

C3.2.2: How to reduce contract transactions fee of blockchain?

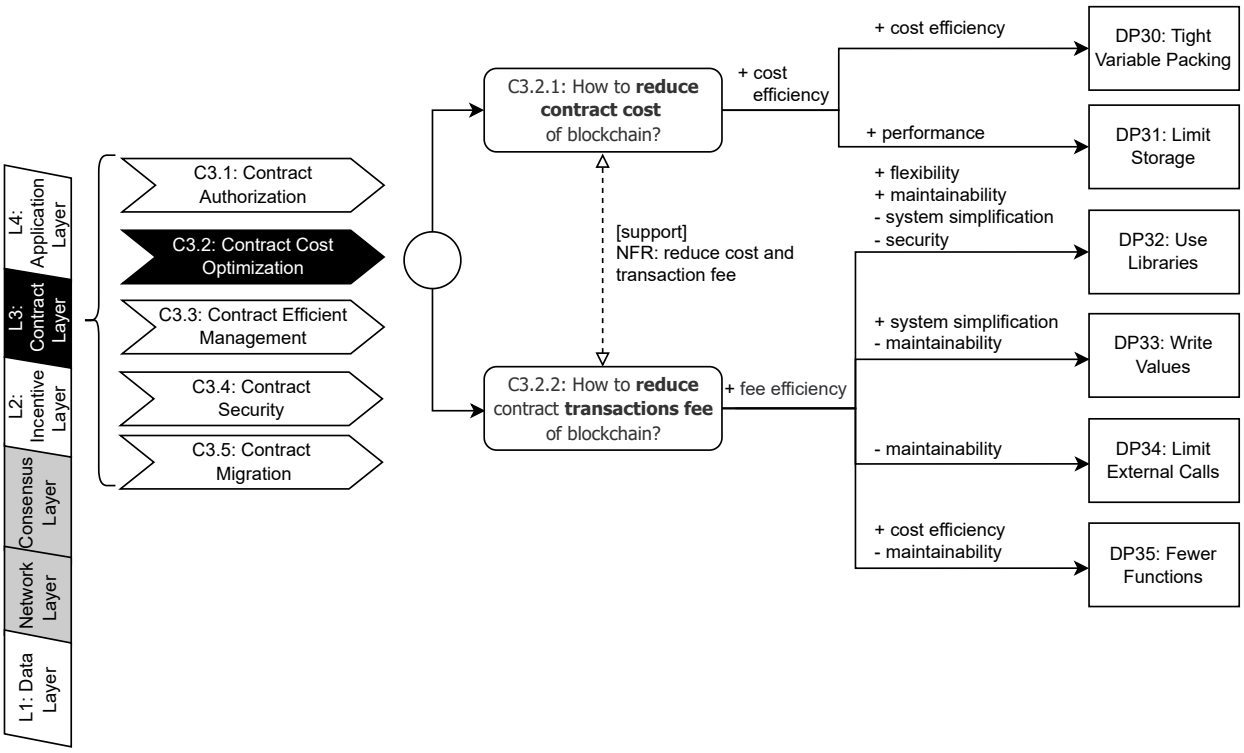

Fig. 8: Decision Model for Contract Cost Optimization (C3.2)

TABLE 9: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C3.2: Contract Cost Optimization)

ID	Pattern Name	Quality Attribute	Impact	Quality Attribute Mapping Description
DP30	Tight Variable Packing	Cost Efficiency	Positive	Reduces on-chain costs by maintaining compactness of constant strings and minimize on-chain data by storing static variables.
DP31	Limit Storage	Performance	Positive	Improves performance by minimizing read and write operations with the blockchain database, reducing overhead.
DP32	Use Libraries	Flexibility	Positive	Enhances flexibility by calling external libraries to assist with non-blockchain tasks, reducing on-chain overhead.
		Maintainability	Positive	Improves maintainability by calling external libraries that are easier to update than on-chain smart contracts.
		System Simplification	Negative	External libraries lead to high system complexity and calling them is more costly.
		Security	Negative	Compromises security due to the vulnerabilities and limitations of off-chain libraries, such as data opacity and behavior inconsistency.
DP33	Write Values	System Simplification	Positive	Simplifies on-chain logic and reduces complexity by directly writing final values.
		Maintainability	Negative	Reduces maintainability as direct value use can result in incomplete code logic, making it difficult to troubleshoot and upgrade.
DP34	Limit External Calls	Maintainability	Negative	Reduces maintainability due to increased complexity from aggregated functions in a single contract.
DP35	Fewer Functions	Cost Efficiency	Positive	It reduces the number of functions in a smart contract, simplifying internal calls and communication.
		Maintainability	Negative	Reduces maintainability as complex, tightly coupled functions can be difficult to test and update.
-	Commonality (DP30 to DP31)	Cost Efficiency	Positive	Both of these patterns reduce on-chain overhead in smart contracts, and avoid excessive resource usage. These patterns are recommended because they are similar to traditional programming practices without too many drawbacks.
-	Commonality (DP32 to DP35)	Fee Efficiency	Positive	All patterns reduce on-chain transaction fees while minimizing blockchain overhead. However, tradeoffs related to maintainability and complexity should be carefully considered.

DP32: Use Libraries. This pattern involves calling external libraries to assist with tasks not inherent to the blockchain, thus reducing on-chain overhead [14]. With the help of a library, blockchain can be applied in a wide range of situations and cope with changes, resulting in higher levels of flexibility. In addition, this pattern has better maintainability as it uses external libraries as off-chain components, which are easier to maintain and update than smart contracts on-chain. However, external libraries lead to high system complexity and calling them is more costly. The process of calling external libraries is also vulnerable to external attacks, and leads to other limitations such as data opacity, and behavior inconsistency due to the uncontrollable off-chain library, resulting in compromised security. Therefore, this pattern has more tradeoff attributes when dealing with simple tasks and is more suitable for high-complexity computation.

DP33: Write Values. This pattern omits the use of functions on smart contracts to compute values during initialization and instead writes values directly [14]. This can simplify on-chain logic and reduce complexity. However, the direct use of final values may result in incomplete code logic, making it difficult to troubleshoot errors and upgrade functions in the future, leading to poor maintainability.

DP34: Limit External Calls. This pattern imposes limitations on the calling of external smart contracts to reduce expensive calling fees, and suggests the use of a multi-purpose contract instead of making multiple calls to single-purpose smart contracts [14]. However, this pattern also leads to the aggregation of functions in a single smart contract and increases the complexity of contract design due to the restriction on external invocation. And the implementation also becomes intricate and disordered, making it difficult to update or extend functions, reducing maintainability.

DP35: Fewer Functions. This pattern reduces the number of functions in a smart contract to minimize transaction fees incurred during implementation [14]. But it is still critical to strike a balance as excessively complex functions can make testing cumbersome and compromise security. This pattern can reduce function declaration and implementation, and simplify internal invoke and communication costs, thus reducing runtime costs. However, this method can also lead to intricate and tightly coupled functions, similar to the Limit External Calls pattern, thereby reducing maintainability due to testing and updating challenges. According to [14], these four patterns are all discussed in technical blogs through practical examples.

Commonality from DP32 to DP35: All of these patterns reduce on-chain transaction fees while minimizing blockchain overhead. However, it is important to note that each pattern achieves this goal through a distinct approach. Use Libraries (DP31) and Write Values (DP32) reduce transaction fees by reducing on-chain contract operations, either by introducing external libraries or directly determining the final result. On the other hand, Limit External Calls (DP33) and Fewer Functions (DP34) decrease costs by minimizing the number of contracts and function invocations. It is crucial to consider the tradeoffs associated with each pattern, such as maintainability and complexity, in determining the most suitable approach to contract design.

C3.3: Contract Management and Operation

Smart contracts have distinct responsibilities, allowing them to collaborate on-chain to facilitate business processes. Nonetheless, in large-scale systems with intricate business operations and widespread applications, mismanagement of smart contract creation, execution, and upgrading can result in extra cost and structural disorder. Given that smart contracts are essentially coded running on the blockchain, numerous traditional design patterns and programming principles can be migrated and applied. By treating contracts as objects, the contract layer can be developed to achieve high cohesion and low coupling, thereby enabling efficient management of the creation, execution, and upgrading of smart contracts. This section gathers design patterns aimed at improving smart contract management, as shown in Fig. 9.

C3.3.1 How to create contracts efficiently?

DP36: Contract Factory. The factory contract is stored on-chain for instantiating other similar contracts, which need only be deployed once and can be used to create multiple instances [21]. Similar to the factory pattern in traditional programming, developers can customize smart contracts by passing parameters without deploying them sequentially. Due to the standard template contract, unexpected coding errors caused by humans can be effectively mitigated, leading to improvement in contract development reliability. Moreover, factory contracts can also observe issued smart contracts more effectively by storing the addresses of all generated smart contracts [24], thus leading to better data verifiability. However, it should be noted that this pattern can be resource-intensive and requires extra overhead for contract deployment and invocation when creating new instances [4] [3]. The pattern has been realized in health care applications [23] and business process monitoring [42].

DP37: Abstract Factory. This pattern delegates service responsibilities to an abstract factory contract. Then the concrete factory object inherits the methods from the abstract factory and customizes them to generate accounts for a particular related group or interacting sub-entities [24]. Due to the abstract factory contract's ability to customize specific contract implementations for complex conditions, this pattern offers enhanced flexibility. Furthermore, it provides low coupling for the interface in instances where an organization needs to extend its departmental functionality, which is similar to the traditional programming pattern, thus avoiding runtime if-else decisions and sharing factory contract features. Nevertheless, this pattern introduces an abstract factory contract as an intermediate layer in the generation and instantiation of the concrete contract. Its complexity and high cost render it suitable for large-scale blockchain systems that require frequent and complex functional changes. Sample code for this pattern is available in [23].

TABLE 10: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C3.3: Contract Management and Operation)

ID	Pattern Name	Quality Attributes	Impact	Quality Attribute Mapping Description
DP36	Contract Factory	Reliability	Positive	Unexpected coding errors caused by humans can be mitigated with standard template contract.
		Data Verifiability	Positive	Improves data verifiability by keeping track of all instantiated contracts, allowing for better observation and management of contract instances.
DP37	Abstract Factory	Flexibility	Positive	Allows the creation of customized contract implementations for complex conditions, which is particularly useful in large-scale blockchain systems.
-	Commonality (DP36 and DP37)	Cost Efficiency	Negative	These patterns have more overhead for contract deployment and contract invoke due to the additional mechanisms. And the abstract factory introduces an intermediate layer of indirect addressing.
DP38	Contract Mediator	System Simplification	Positive	It can manage complex interactions between contracts.
		Interoperability	Positive	Improves interoperability by managing contract interactions through a mediator.
		Scalability	Negative	The mediator may become a bottleneck in large-scale systems.
DP39	Data Segregation	Flexibility	Positive	Separates data storage from contract logic, making upgrades easier.
		Scalability	Positive	Allows shared data across multiple contracts, and decreases effort required to expand the number and scale of smart contracts .
		System Complexity	Negative	Additional resources and external calls are needed for managing separated data.
DP40	Flyweight	Flexibility	Positive	Decouples public data from logic ton ensure better flexibility (Compare with DP39).
		Scalability	Positive	Updating a contract won't affect shared data, only specific data is involved (Compare with DP39).
		System Simplification	Negative	Increased communication complexity and additional transaction validation (Compare with DP39).
		Performance	Negative	Take longer to execute because it becomes another transaction that needs to be validated.
DP41	Contract Façade	Usability	Positive	Provide a simplified interface for interacting with complex contracts.
		Extensibility	Negative	Modifications to the interface code violates the open-close principle.
DP42	State Machine	Reliability	Positive	Increases reliability by ensuring deterministic state transitions.
		Extensibility	Negative	Adding new states requires code-level changes, violating the open-close principle.
		System Simplification	Negative	State definition and transition leads to more smart contracts.
DP43	Publisher-Subscriber	Flexibility	Positive	Allows dynamic subscription and publication of relevant information.
		Cost Efficiency	Negative	Adds potential high costs for handling detailed message topics.
		Performance	Negative	Subscribers will receive on-chain information updates with more latency than traditional systems.
DP44	Proxy Contract	Usability	Positive	Allows transparent contract upgrades through proxies.
		Reliability	Negative	Leads potential inconsistencies between proxies and underlying contracts.
DP45	Contract Register	Usability	Positive	Maps human-readable names to contract addresses.
		Reliability	Positive	The Contract Register is uniquely maintained, making disparate behavior impossible and enhancing reliability. (Compare with DP44)
		Extensibility	Negative	The register can't modify the function interface called by other contracts (Compare with DP44, DP46, DP47).
DP46	Contract Composer	Extensibility	Positive	Functionality of smart contract can be easily modified through address pointers.
		Flexibility	Positive	Allows for the flexible combination of contract functions.
		Performance	Negative	Intricate invocation chains lead to higher collaboration latency.
DP47	Contract Decorator	Flexibility	Positive	Different behaviors can be realized through combination of the decorator contracts.
		Scalability	Negative	Results in many similar contracts leading to system complexity and disorder.
-	Commonality (DP36 to DP47)	Maintainability	Positive	Despite added complexity, these patterns improve maintainability by providing structured contract management approaches.
		System Simplification	Negative	All patterns increase system complexity due to added mechanisms for contract management.
		Cost Efficiency	Negative	These patterns generally increase costs related to contract deployment, invocation, and communication.

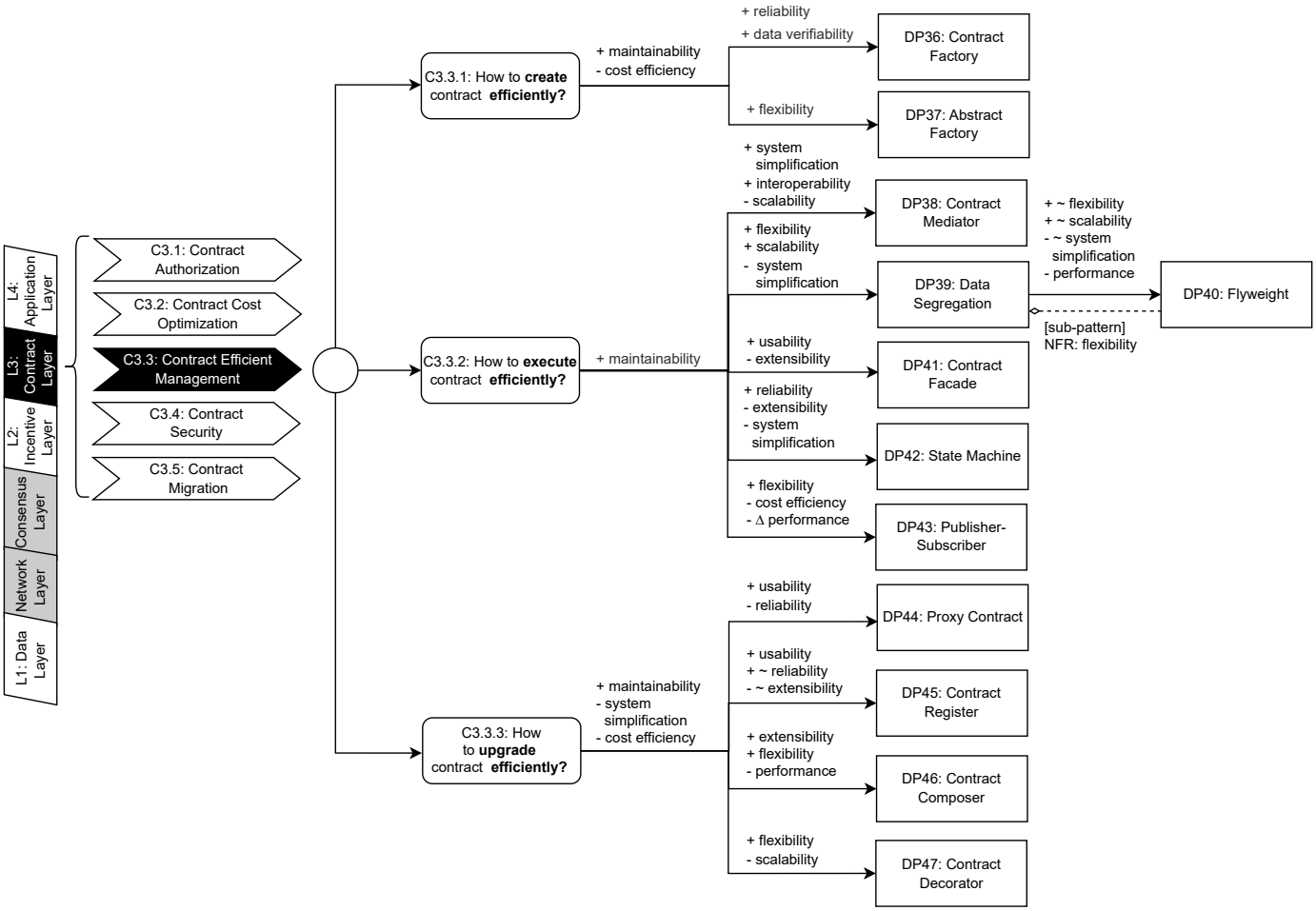

Fig. 9: Decision Model for Contract Management and Operation (C3.3)

Commonality for DP36 and DP37: Both increase system costs. These patterns have more overhead for contract deployment and contract invoke due to the additional mechanisms. And the abstract factory introduces an intermediate layer of indirect addressing. Therefore, in practice, the implementation complexity of both patterns can be increased accordingly.

C3.3.2 How to execute contracts efficiently?

DP38: Contract Mediator. This pattern defines interfaces to communicate with the peer contract for collaboration. The mediator contract encapsulates the interactions and invocations and thus contracts are loosely coupled, which reduces the complexity of invocation dependency and communication [21]. Interoperability can be enhanced by leveraging the mediator contract to manage complex dependencies. However, as the system scales, the mediator contract will be disordered and complex, making it difficult to support large-scale applications and causing poor scalability. The case of this pattern is also discussed in [21].

DP39: Data Segregation. In this pattern, the data store is isolated from logic (or data operation), and thus avoids transferring data during contract logic upgrades. By following the separation of concerns principle, this pattern facilitates the decoupling of data and contract logic, enabling better flexibility [3] [25]. In addition, if the data can be clearly separated and generalized, common data can be shared by all relevant smart contracts, achieving increased storage efficiency. Moreover, the effort required to expand the number and scale of smart contracts decreases, especially for those with similar functions and data requirements, realizing better scalability. However, due to the separation of logic and data, more resources and external calls are needed and the complexity of the system increases [4]. In practice, projects like Chronobank[42] and Colony[43] introduce the data contract as a generic data structure used by all the other logic smart contracts.

DP40: Flyweight. This pattern builds on Data Segregation principles, offering additional data separation and grouping mechanisms. Common data for a group of clients is stored in a shared contract, while private data is kept in specific contracts for each client. Flyweight reduces redundancy, optimizes memory consumption, and decouples public data from logic, providing superior flexibility than Data Segregation [4]. Updating a contract won't affect shared data, and new contracts only need to focus on their own private data, achieving better scalability for large-scale systems. However, the Flyweight pattern increases communication complexity and contract numbers, leading to a more intricate system than the Data Segregation pattern. Besides, each transaction may take longer to execute because it becomes another transaction that

needs to be validated, resulting in low performance. A blockchain-based healthcare app proposed in [23] adopted this pattern to manage common data and individual data.

DP41: Contract Façade. This pattern handles contract addresses by offering developers a unified and simple interface in the form of a smart contract. This approach simplifies operation and invocation, improving usability for users [21]. This pattern can also manage complicated business logic implemented in separate contracts [24], improving maintainability. However, upgrading functions may require modifying the interface code, which violates the open-close principle resulting in low extensibility. [21] This pattern has been applied in a real case based on the blockchain traceability system [62].

DP42: State machine. This pattern executes contract operations based on the current state, and contracts can have different behaviors in different states. States are cohesive and can be easily managed and tested, and allow for introducing updated functionality through state transfers for better maintainability [6], [22]. New functionality can be introduced through state transfers rather than redundant modification of conditional transition statements like if-else, thus achieving better maintainability [26]. In addition, function calls trigger predefined and deterministic state transfers to ensure system reliability. However, it does not support the open-close principle, and adding new states to contracts requires modifying the code of state transfers, compromising extensibility. System complexity increases with state definition and transition, leading to more smart contracts. A sample code for a deposit lock contract using the State Machine is provided in [6] and [7].

DP43: Publisher-Subscriber. This pattern spreads information only to interested clients that subscribe to related events or topics. This pattern decouples the relationship between publisher and subscriber and can facilitate scalable information filtering and alleviate the redundant communication process [21] [25]. Similar to the observer pattern in traditional programming, smart contracts can dynamically publish or subscribe to relevant information, enhancing flexibility. However, detailed message topics or numerous client requests may incur significant costs [24]. In addition, complex dependencies will be formed and subscribers will receive information updates with more latency than traditional systems. A possible optimized solution is to use broader topics with fewer on-chain filters, and detailed filters can be handled off-chain. This pattern has proven effective in a healthcare system [23] and traceability system [21], achieving better interaction.

C3.3.3 How to upgrade contracts efficiently?

DP44: Proxy Contract. This pattern creates proxies to store contract addresses and forward requests to the latest SC version and the upgrade of contract is well maintained by the proxies [6], [14]. Simple data can be stored in the proxy for quick queries, and complex queries are executed by the proxy on-chain. It follows the same interface as the real object and can execute the original function implementation as required. Therefore, the proxies are transparent to the user, and the same is true of the original contract modification, achieving better usability for users and developers [4], [5]. Moreover, there are multiple proxies dealing with requests concurrently, and requests for data can be responded to faster. However, the system has a relative high probability of exceptions caused by proxy contracts, thus resulting in reliability reduction. The Proxy pattern introduces an intermediate layer, leading to possible inconsistencies between external calls and internal implementation. More specifically, when a client accesses the real object directly and other clients access the proxy, disparate behavior may be caused [25]. For example, the versions of SC addresses stored in the proxy are not updated in a timely manner, leading to different access results for addresses. A case of blockchain-based healthcare app is provided in [23], and [6] provides the sample code.

DP45: Contract Register. The pattern maintains a mapping between human-readable names and hexadecimal blockchain addresses of registered contracts in a registry. This registry serves as a pointer to track the current version of a contract via its address [4], [5]. By replacing hexadecimal addresses, readable name-based dependencies can be preserved and remain transparent to users, significantly improving usability [3]. Unlike the Proxy Contract pattern, the Contract Register is uniquely maintained, making disparate behavior impossible and enhancing reliability. However, maintaining the mapping between contract names and addresses requires additional resources. This pattern also has less extensibility than the other three patterns because it cannot modify the function interface called by other contracts [3]. In Ethereum, the Ethereum Name Service (ENS)[44] has emerged as a solution for timely pointing to the latest contract version [6]. In other cases, the Truffle framework[45] provides built-in functionality for contract registration, and an in-browser application called Regis[46] can also serve as a contract registry service.

DP46: Contract Composer. This pattern can construct a complex structure from multiple small modules implemented by smart contracts, which is a form of componentization [7]. Through address pointers, one contract can invoke another, and functional modifications can be made simply by altering the corresponding addresses [6]. This component-based approach enhances system extensibility. This pattern also allows for the flexible combination of contract functions to satisfy the complex and changing needs of businesses [21], similar to microservices, while offering superior flexibility. However, as the number of contracts increases, the Contract Composer pattern can result in more complex communication, with intricate invocation chains. This method may lead to higher collaboration latency, which is resource costly and adversely affects performance [6], [21]. The sample code is provided in [6], and a real-world blockchain-based traceability system also adopts the pattern [62].

DP46: Contract Decorator. This pattern can encapsulate the old contract and attach the necessary features to the updated version when there is a change in requirement. This is to avoid the rewriting and rebuilding of the whole contract when changes are made [21]. A variety of different behaviors can be realized to meet the requirements through combination of the decorator contracts, leading to improved flexibility. However, the use of this pattern on a large-scale system may result in many similar contracts leading to system complexity and disorder, as commonly observed in traditional programming.

As a result, this pattern causes maintenance difficulty for large-scale applications, and makes them less scalable. [21] discuss the case of this pattern.

Commonality from DP44 to DP47: All of them result in increased system complexity and system costs. All four patterns realize certain features or mechanisms by increasing the number of contracts. And the coding and deployment of these smart contracts, as well as the design of dependencies, can significantly increase the static complexity of the system. Moreover, with the introduction of mechanisms and numerous contracts, it leads to more resource consumption on the blockchain with the overhead of inter-contract communication, leading to more cost at runtime. Similarly, due to occupying more computational and storage resources on the blockchain, coupled with the complicated contract call chain, transaction fees will be increased accordingly.

C3.4: Contract Security

Compared to traditional software, smart contracts offer better security due to their decentralized nature. However, they are also exposed to different types of risks and attacks, such as double-spend and re-entrancy attacks, which target their vulnerabilities [27]. Unlike traditional software, smart contracts cannot take immediate action or be terminated right away once malicious behavior is detected. Therefore, it is crucial to prevent attacks beforehand and respond quickly in case of an attack. In this layer, we present and analyze design patterns that can reduce risks, prevent attacks, and terminate contracts, with the aim of enhancing blockchain security, as shown in Fig. 10

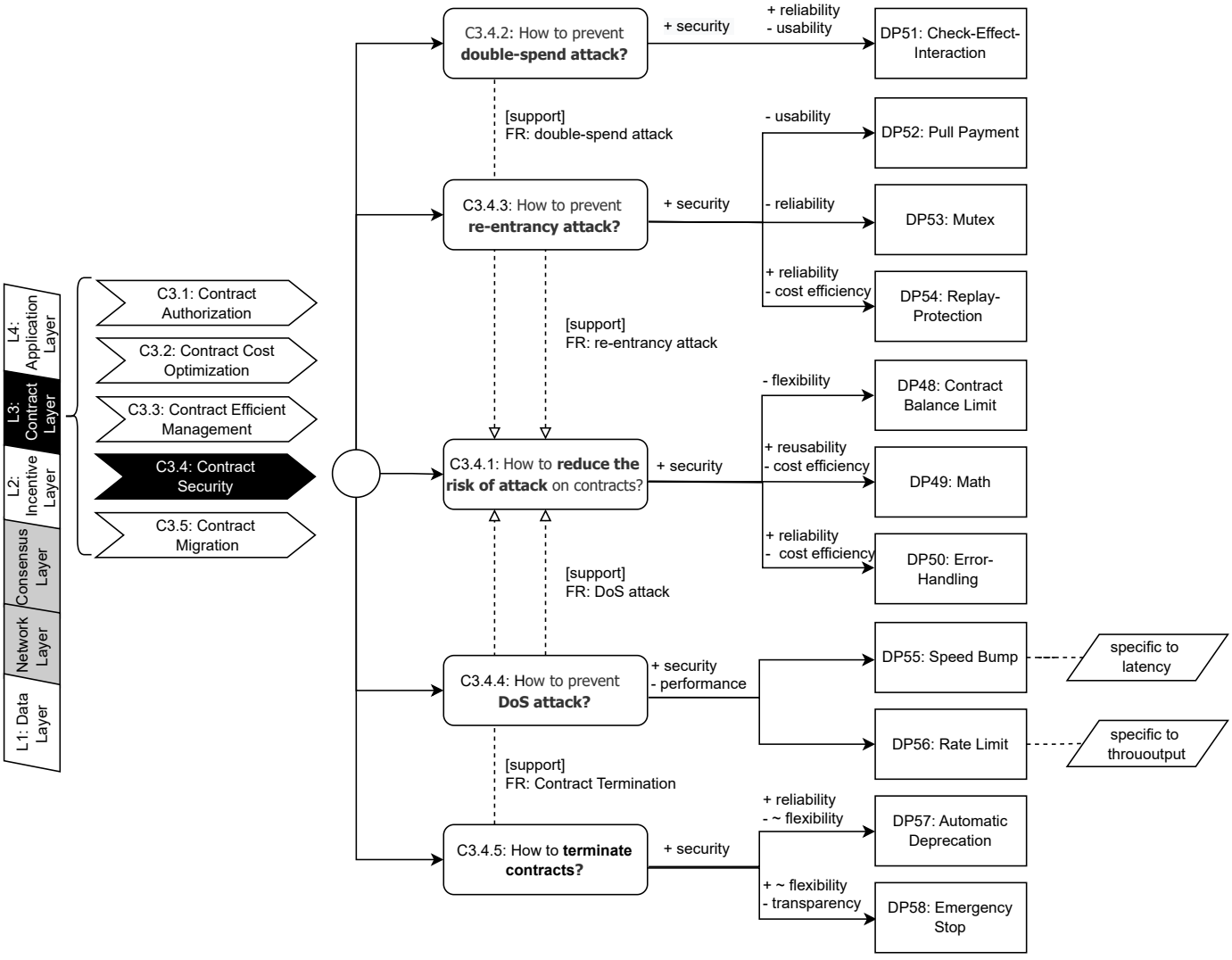

Fig. 10: Decision Model for Contract Security (C3.4)

C3.4.1 How to reduce the risk of attack on contracts?

DP48: Contract Balance Limit. This pattern restricts the maximum amount of funds stored in a contract to prevent overconcentration of assets. When the limit is reached, any further calls are rejected, unless provisions for funds sending exist [4]. This pattern can disperse asset targets, providing better security as attacks cannot damage all assets at once.

TABLE 11: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C3.4: Contract Security)

ID	Pattern Name	Quality Attributes	Impact	Quality Attribute Mapping Description
DP48	Contract Balance Limit	Flexibility	Negative	Caps the funds stored in a contract, restricting fund flows.
DP49	Math	Reusability	Positive	Encapsulates common operations in a math library for relevant contracts to call.
		Cost Efficiency	Negative	Leads to additional resource consumption due to the implementation and dependency on extra libraries.
DP50	Error-Handling	Reliability	Positive	Uses exception handling mechanisms to improve implementation soundness.
		Cost Efficiency	Negative	Implementing error handling leads complexity and reduce resource efficiency.
DP51	Check-Effect-Interaction	Security	Positive	Ensures security by checking conditions before operations, preventing attacks and unexpected errors.
		Usability	Negative	Changing state before calling a function is counter-intuitive.
DP52	Pull Payment	Security	Positive	Reverse the payment process, allowing users to withdraw their funds themselves during transactions to avoid deliberate sabotage.
		Usability	Negative	Reduces usability due to the reverse payment process, which requires individual calls and is counter-intuitive [24].
DP53	Mutex	Security	Positive	Use a mutex lock to prevent concurrent calls, reducing re-entrancy risks.
		Reliability	Negative	implementing mutex locks can bring programming flaws like incorrectly locking certain contracts or even causing malicious deadlocks.
DP54	Replay-Protection	Security	Positive	Use digital signatures to prevent re-entrancy transactions and reduce parameter errors.
		Reliability	Positive	Enhances reliability by preventing re-entrancy transactions and reducing parameter errors.
		Cost Efficiency	Negative	Additional signature and verification mechanisms lead to increased resource consumption and system costs.
DP55	Speed Bump	Security	Positive	Increases security by slowing down sensitive tasks, allowing time to respond to potential attacks.
		Performance	Negative	Reduces performance by increasing transaction latency.
DP56	Rate Limit	Security	Positive	Enhances security by limiting the rate of task execution, reducing the risk of DoS attacks.
		Performance	Negative	Lowers performance by reducing transaction throughput.
DP57	Automatic Deprecation	Security	Positive	Automatically terminates contracts when needed.
		Reliability	Positive	Enables automatic termination of smart contracts without external intervention during emergencies.
		Flexibility	Negative	Reduces flexibility as termination conditions are predefined (Compare with DP58).
DP58	Emergency Stop	Security	Positive	Allows authorized entities to stop contracts during emergencies.
		Flexibility	Positive	Improves flexibility with a manual override mechanism (Compare with DP58).
		Transparency	Negative	Reduces transparency by centralizing the power to stop contracts.
-	Commonality (DP48 to DP58)	Security	Positive	All patterns enhance security by addressing specific threats like double-spend, re-entrancy, DoS attacks, and contract termination.

However, this restriction partially limits funds flow and compromises flexibility. The blockchain game CryptoKitties[47] adopted this pattern on the Ethereum main-net.

DP49: Math. This pattern protects the correctness and security of critical operations, such as using a "Math" library to prevent exceptions or malicious behaviors like integer overflows [8]. Typically, such a library is implemented using smart contracts and encapsulates numerous common operations for other relevant contracts, thereby enhancing their reusability. However, this pattern leads to additional resource consumption due to the implementation and dependency on extra libraries, resulting in more costs. An example is the Ethereum Contract Badge[48], which incorporates a method called subtractSafely to prevent negative funds.

DP50: Error-Handling. This pattern recommends that all functions should have a return value to report the operation status. This ensures successful execution can be determined [63], [64]. This pattern uses exception handling mechanisms to improve implementation soundness, reduce errors and exceptions, and increase reliability and security. However, implementing error handling requires additional code, which can be complicated and reduce resource efficiency, leading to increased operating overhead and transaction fees. One practical example of this pattern is Vyper, which automatically checks return values [65].

C3.4.2 How to prevent double-spend attacks?

DP51: Check-Effect-Interaction. This pattern needs to check the necessary conditions before operations. First check whether the necessary conditions are met (check), then change contract state (effect), and finally make contract calls (interaction) [6], [27]. Ideally, interactions with other contracts should be deferred until all other steps are completed, and malicious access cannot enter the same function in a previous state [20], [66]. Since the necessary conditions are

checked before each call, unexpected errors and potential attacks can be avoided, ensuring reliable and secure transactions. However, the process of changing state before calling a function is counter-intuitive since it is customary to wait for a function call to execute successfully before making any state or variable changes [4]. As a result, the usability may be compromised. [27] provides a sample code of this pattern.

C3.4.3 How to prevent re-entrancy attacks?

DP52: Pull Payment. This pattern allows for a reversal of the payment process, allowing users to withdraw their funds themselves during transactions to avoid deliberate sabotage [6]. For example, a transaction where User A sends 5 ETH to User B can be reversed so that User B withdraws 5 ETH from User A. This protects User A's assets, as it prevents malicious repeated calls to the token-sending function on their account and makes it impossible for User B to withdraw twice. This pattern can also be applied to other unbounded data operations [67]. However, the reverse process can be counter-intuitive and decrease application utility due to individual calls to smart contracts for each account [24], which can lead to low usability. Cryptopunks[49] is one example that uses this pattern, and [68] provides a sample implementation of the pattern.

DP53: Mutex. This pattern locks the state of smart contracts using a mutex lock when the contract is called to prevent concurrent external calls [4]. Due to mutex locks, malicious re-entrancy calls will be aborted [24]. Contract code execution in the parent contract is also prevented [27]. Therefore, the probability of a re-entrancy attack is reduced. However, implementing mutex locks is not a straightforward solution for preventing re-entrancy attacks and may result in additional risks, especially in cases of multiple function calls [24]. Such risks can include programming flaws like incorrectly locking certain contracts or even causing malicious deadlocks, thereby reducing system reliability. The resources with mutex locks can only be accessed serially, but this isn't a major drawback since there is probably a better solution when mutex delays task execution [4]. A security plugin for locking smart contracts and preventing nested function calls is provided in [26].

DP54: Replay-Protection. This pattern employs digital signatures to authenticate identity by signing all transaction parameters and the current nonce. Once the target smart contract verifies the transaction, the nonce is changed, rendering future transactions with the same signature invalid [65]. This pattern enhances security and reliability by preventing re-entrancy transactions and reducing parameter errors. It is also suitable for protecting assets during hard forks on public chains, like Ethereum. However, the additional signature and verification mechanisms required for this pattern can lead to increased resource consumption and system costs, with varying impacts depending on the signature algorithm used. The Hyperledger Fabric, a consortium blockchain solution, has implemented this mechanism in its nodes.

C3.4.4 How to prevent DoS attacks?

DP55: Speed Bump. This pattern is used to deliberately slow down the execution time of sensitive tasks, providing more time to detect and respond to malicious behavior, and preventing system crashes caused by repeated invocations of sensitive contracts [7], [27]. This pattern differs from the Rate Limit pattern in that it is an inherent feature specifically designed to limit transaction latency. An example contract implementing a withdrawal delay using the Speed Bump pattern is provided in [27] and [7].

DP56: Rate Limit. This pattern limits the maximum rate of task execution, preventing users from calling a specific contract too frequently and thereby reducing the likelihood of malicious blocking [7], [27]. This pattern is different from the Speed Bump pattern and limits transaction throughput. An example of the Rate Limit pattern that prevents excessively repetitive function execution is also provided in [27] and [7].

Commonality for DP55 and DP56: Both Speed Bump and Rate Limit patterns lead to degradation of system performance. The former increases transaction latency while the latter reduces throughput intentionally. Despite the performance impact, blockchain systems can be safer.

C3.4.5 How to terminate contracts?

DP57: Automatic Deprecation. This pattern defines the way of stopping SCs, either by manually inserting the ad-hoc code or by automatically calling the suicide function. This pattern is more reliable as it enables automatic termination of smart contracts without external intervention upon the occurrence of fatal exceptions or upon the elapse of a specified time period [6]. This pattern offers better security and prevents potential asset loss by managing the executability of smart contracts [20]. However, because this method requires predefined conditions before smart contract deployment and cannot be changed once started, making it less flexible compared with the other pattern. In practice, one of the most popular cross-chain projects, Polkadot[50], applies this pattern to disable contracts.

DP58: Emergency Stop. This pattern integrates emergency stop functionality into the contract to provide authenticated entities with the ability to forcibly stop SC execution, especially for the sensitive functions [7]. Compared with Automatic Deprecation, he Emergency Stop pattern can manually disable the contract at any time by force, which is more flexible. However, due to security considerations, not everyone can conduct the emergency stop operation on the contract. The operation is a special privilege reserved only for authorized parties, and its use may lead to centralization and reduced transparency. So it is crucial to choose the right authorized party, and multi-party authorization can be introduced to prevent centralization [4]. A sample code implemented using Solidity is provided in [27] and [7].

Commonality from DP48 to DP58: All patterns improve security, and this section organizes various design patterns for different security issues. Initially, we discussed general methods for preventing attacks, including asset restriction, an essential operations repository, and error-handling mechanisms. Next, we presented three specific solutions to address distinct types of attacks, namely double-spend, re-entrancy, and DoS attacks. In the event of an attack, disabling the contract is necessary. Therefore, integrating an emergency stop feature into smart contracts is crucial.

C3.5: Contract Migration

As previously mentioned in C1.4, blockchain migration is essential to address rapidly changing requirements. It is essential to migrate not only blockchain data but also blockchain business logic. That is, the migration of smart contracts on the blockchain. This section will focus on the design patterns for contract migration, specifically how to execute smart contracts on a different blockchain, as shown in Fig. 11.

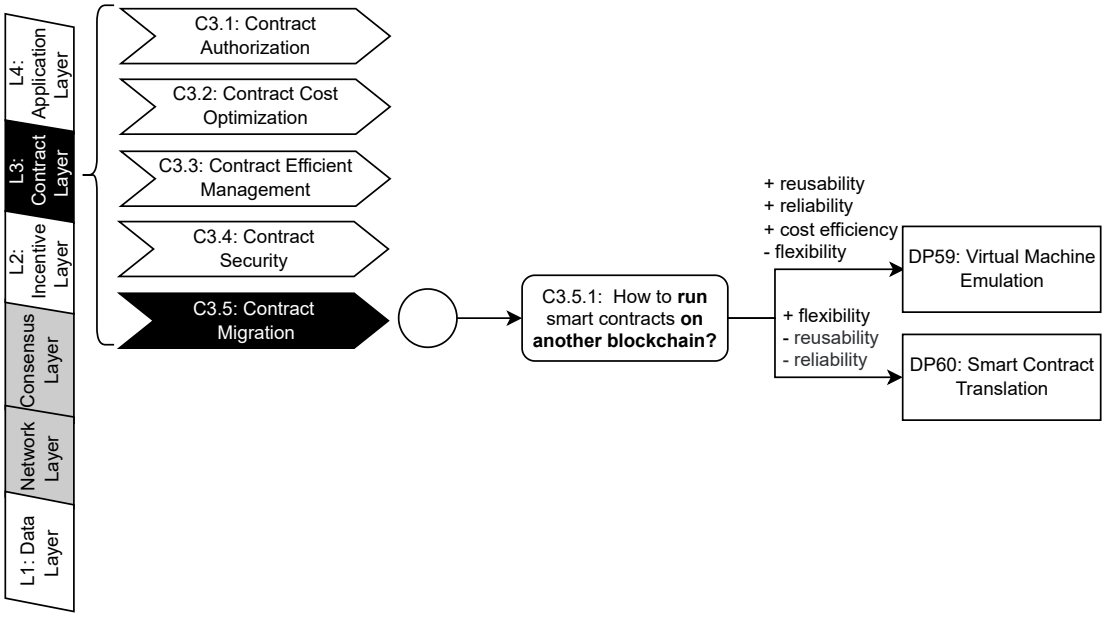

Fig. 11: Decision Model for Contract Migration (C3.5)

TABLE 12: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C3.5: Contract Migration)

ID	Pattern Name	Quality Attribute	Impact	Quality Attribute Mapping Description
DP59	Virtual Machine Emulation	Reusability	Positive	Allows direct execution of contract code from another blockchain, eliminating the need for code translation and testing.
		Reliability	Positive	Source code is not modified, the original behavior and correctness can be well-preserved.
		Cost Efficiency	Positive	Bytecode-level formal verification can be used to verify compatibility, further reducing smart contract validation costs.
		Flexibility	Negative	Reduced flexibility due to the need for the target blockchain to fully simulate the source blockchain's smart contract environment.
DP60	Smart Contract Translation	Flexibility	Positive	Tthe translated smart contract can be deployed on any turing-complete target blockchain, without strict application requirements.
		Reusability	Negative	The translated contract code may not have the exact same features and behaviors as the original, requiring extensive testing and adjustments.
		Reliability	Negative	Results in challenges in ensuring that the translated code behaves identically to the original.

C3.5.1: How to run smart contracts on another blockchain?

DP59: Virtual Machine Emulation. This is a pattern that reproduces the smart contract environment on the target blockchain, allowing for the direct execution of contract code from another blockchain [11]. Additionally, bytecode-level formal verification can be used to verify compatibility, further reducing smart contract validation costs and complexity [69]. This approach eliminates code translation and testing, and smart contracts become more reusable. Furthermore, since the source code is not modified, the original behavior and correctness can be well-preserved, ensuring contract code reliability on the target blockchain. However, the target blockchain must fully simulate the smart contract operating environment of the source blockchain, leading to strict application conditions and reduced flexibility. As an example of this pattern in practice, Deloitte migrated smart contracts from Ethereum to VeChain because both blockchains support the same Ethereum Virtual Machine (EVM).

DP60: Smart Contract Translation. This pattern translates SC code manually and deploys it on any turing-complete target blockchain. Due to reverse engineering of the code, there is no strict application requirement and better flexibility. However, translating smart contracts can be challenging and it is difficult to ensure that the translated contract code has the exact same features and behaviors as the original. Therefore, extensive testing and adjustments are required, resulting in poor reusability and reliability. To save time and effort, automated translation and testing tools are recommended. In

practice, Counterparty and Tron have enabled Ethereum smart contracts on Bitcoin and Tron, respectively, by modifying the original Solidity code.

Commonality for DP59 and DP60: Both patterns are commonly used for smart contract migration on Ethereum or Bitcoin. The Virtual Machine Emulation pattern (DP60) is preferred when environment simulation is not too difficult [11]. On the other hand, the Smart Contract Translation pattern (DP61) is more suitable for cases where the contract code is simple. The choice between these patterns depends on the amount of effort required for the migration process.

L4: Application Layer

Application layer focuses on Decentralized Applications (DApp). This layer serves the users directly and shields the differences among blockchain implementations. Compared with traditional software, DApps need to support digital identity and entity cooperation in a decentralized landscape, and consequently enrich the blockchain ecosystem. In this layer, various design patterns related to blockchain applications are collected and analyzed, including identity management and verification, as well as business cooperation.

C4.1: Identity Management and Verification

Blockchain technology has emerged as a popular solution for identity management due to its ability to address traditional challenges such as information leakage and complex verification processes [11]. These solutions, also known as self-sovereign identity solutions, are becoming increasingly popular. However, in a decentralized environment such as blockchain, effective identity management and verification can become even more complex and challenging. Therefore, it is necessary to summarize relevant design patterns to address typical challenges and better manage user identity in a decentralized manner. This section aims to collect relevant patterns and analyze their quality attributes, as shown in Fig. 12.

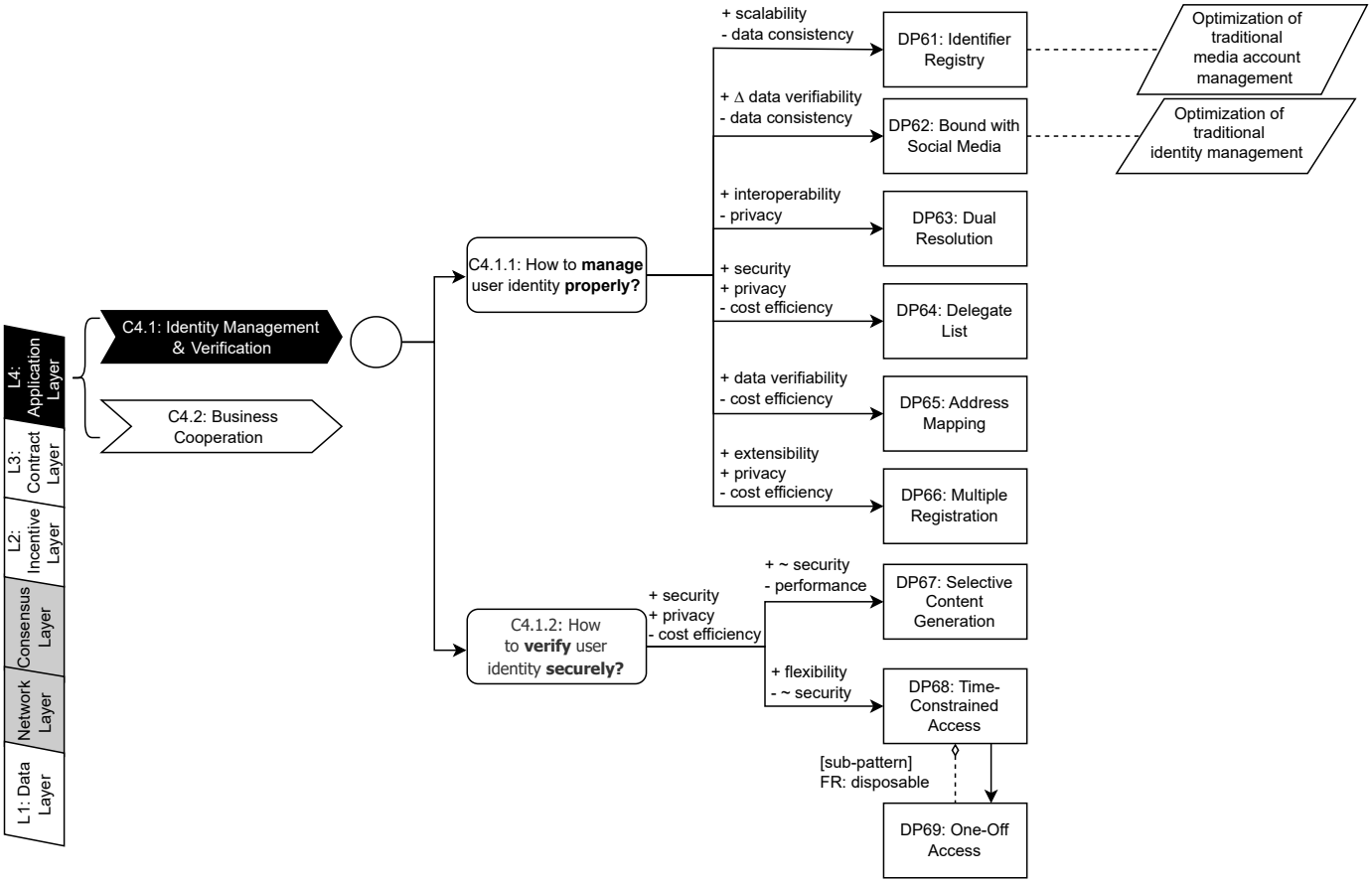

Fig. 12: Decision Model for Identity Management and Verification (C4.1)

C4.1.1: How to manage the user identity properly?

DP61: Identifier Registry. This pattern is a decentralized solution for identity management that employs smart contracts as registries to maintain the association between on-chain identifiers and off-chain identity attributes [11]. As an inherent attribute, this pattern is an optimized version of traditional identity management. The identifier used is also known as a Decentralized Identifier (DID) in some cases. The DID is a globally unique series stored persistently in smart contracts and can be used to locate an entity such as a human, organization, or device within a particular scenario, as well as retrieve

TABLE 13: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C4.1: Identity Management and Verification)

ID	Pattern Name	Quality Attribute	Impact	Quality Attribute Mapping Description
DP61	Identifier Registry	Scalability	Positive	Uses simple data structures to store numerous identifiers with minimal storage space usage, making it suitable for large-scale applications.
		Data Consistency	Negative	Consistency is difficult to guarantee as off-chain data modification cannot be prevented, similar to Off-chain Data Store pattern.
DP62	Bound with Social Media	Data Verifiability	Positive	Blockchain is utilized as an authoritative source for social media endorsement, optimizing traditional media account management.
		Data Consistency	Negative	Data consistency may not always be guaranteed as the mapping relationship on-chain may not reflect off-chain media data modifications.
DP63	Dual Resolution	Interoperability	Positive	Ensures efficient and reliable verification and interaction by resolving each other's identities before formal interaction.
		Privacy	Negative	Increases the risk of privacy leaks as more identity information is shared during the resolution process.
DP64	Delegate List	Security	Positive	Allows the replacement of the private key when lost, preventing information leakage due to external attacks.
		Transparency	Positive	Maintains transparency in the restricted application domain despite the centralized tendency of using trusted trustees.
		Cost Efficiency	Negative	Requires considerable storage space for maintaining and updating the delegate list, leading to additional computational overhead.
DP65	Address Mapping	Data Verifiability	Positive	Improves data verification by managing associations between smart contracts and user addresses, enabling independent tracking of interactions.
		Cost Efficiency	Negative	Incurs additional costs due to computation and storage overhead, especially in scenarios with complex business and invocation dependencies.
DP66	Multiple Registration	Extensibility	Positive	Generates unique identifiers for each user in each different contexts, with loss or gain of one identifier in a certain context not affecting others.
		Privacy	Positive	Protects personal privacy by maintaining numerous different identities, making it difficult to correlate between identifiers.
		Cost Efficiency	Negative	Higher costs associated with maintaining and updating multiple identities.
DP67	Selective Content Generation	Security	Positive	The verification contents are restricted to the minimum necessary, reducing the risk of information leakage (Compare with DP68, DP69).
		Performance	Negative	Negatively impacts runtime performance due to the need to generate different credentials for different validators.
DP68	Time-Constrained Access	Flexibility	Positive	Provides better flexibility by generating time-constrained links for verifiers, ensuring secure and temporary access to user identity.
		Security	Negative	A snapshot may be maliciously created when access is available. Thus, the historical identity state can be leaked, leading compromised security (Compare with DP67).
DP69	One-Off Access	Flexibility	Positive	Offers flexibility similar to Time-Constrained Access by providing one-off access links for secure verification.
		Security	Negative	Similar to Time-Constrained Access, a snapshot may be maliciously created, leading compromised security (Compare with DP67).
	Commonality (DP67 to DP69)	Privacy & Security	Positive	All patterns enhance privacy and security by minimizing exposed data or restricting access duration, reducing the attack surface and potential for malicious behavior.
		Cost Efficiency	Negative	All patterns lead to increased system costs due to the need for additional communication complexity, generation of credentials, and maintenance of temporary access mechanisms.

their identity attribute data [11]. This record is kept in a simple data structure and only stores the identifiers and locations of identity attributes [11], resulting in minimal storage space usage and support for storing numerous identifiers in large-scale applications, thus achieving better scalability. However, similar to the Off-chain Data Store patterns (DP8, DP9), this method cannot prevent off-chain data modification and cannot recover once modified, making consistency difficult to guarantee. Realistic projects such as Sovrin and Jolocom, implement distributed identity systems and registers using blockchain contracts.

DP62: Bound with Social Media. This pattern establishes a one-to-one mapping relationship between social media and blockchain identity, enabling them to mutually verify each other's authenticity [11]. Using blockchain as an authoritative source for social media endorsement, this pattern optimizes traditional media account management. In comparison to traditional systems, media data stored on-chain is immutable and easier to track and verify, thereby increasing data verifiability. Moreover, transparency is not compromised as it is based on the trustworthiness of trusted participants. However, data consistency cannot always be guaranteed as the mapping relationship on-chain may not reflect off-chain media data modifications. In Onename, users can easily register a blockchain ID and link it to their existing social media accounts like Twitter, Facebook, Github, etc.

DP63: Dual Resolution. This pattern requires that two blockchain entities in a relationship must first resolve each

other's identities and obtain the necessary information for verification and communication before any formal interaction takes place. The entity that has already interacted can be recorded and tracked for the next communication [11]. By keeping the relevant information, efficient and reliable verification and interaction can be ensured, leading to better interoperability. However, this pattern comes with the risk of privacy leaks, as more identity information is shared with others. Although this pattern is not directly mentioned in applications, users need to resolve each other's DID when interaction is possible [11].

DP64: Delegate List. This is a pattern where the identity owner selects a group of reliable trustees to help in changing ownership, updating identity information or freezing the lost identity if needed [11]. Since trusted trustees are recognized as credible, no decrease in transparency occurs in the restricted application domain despite the centralized tendency. And the private key can be replaced when lost to avoid information leakage caused by external attacks, which brings better security. However, maintaining and updating a delegate list requires considerable storage space. Additionally, every request for identity and private key recovery is a single transaction, resulting in computational overhead and additional transaction fees [11]. In practice, Sovrin and Baidu Cloud have implemented a secret key recovery mechanism with a group of delegated entities.

DP65: Address Mapping. This pattern can establish and maintain mappings between user addresses and smart contracts, thus the user who interacts with a certain contract can be tracked independently [5]. This pattern can help manage associations between smart contracts and user addresses for relevant data verification. However, the mapping between contracts and user addresses may incur additional costs. Especially in applications with complicated business and invocation dependencies, the computation and storage overhead may be significant. In practice, the Solidity programming language offers built-in support for address mappings.

DP66: Multiple Registration. This pattern allows for the generation of unique identifiers for each user in various contexts, such as a student number or identity card number. The Master and Sub Key Generation pattern is useful as it provides sub-keys to register distinct identities [11]. It has better extensibility because the identifiers are highly cohesive, and loss or gain of one identifier does not affect other identities. As it is difficult to correlate between identifiers, personal privacy can be protected. However, Multiple Registration maintains numerous different identities for each user with registration and updates, and has a higher cost than a single identifier. Projects such as Blockstack and DAML enable entities to have multiple identifiers to represent different identities.

C4.1.2: How to verify the user identity securely?

DP67: Selective Content Generation. This pattern allows the user to determine which identity attributes are included in the credentials for validation by the counterparty, and the content is usually supposed to be within the minimum range to meet the validation requirements [11]. By restricting verification content, this pattern reduces the risk of unnecessary information leakage, and thus enhances security and privacy. However, since different verifications may require different data, the user needs to generate different credentials for different validators, which can be inefficient and negatively impact runtime performance [11]. This can also lead to increased resource and communication costs. In practice, uPort and Sovrin integrate cryptographic mechanisms using this pattern to ensure secure and reliable digital identity management.

DP68 and DP69: Time-Constrained Access and One-Off Access. The Time-Constrained Access Pattern generates a time-constrained, identifiable link for the verifier. The verifier must complete the verification of the user's identity within the predefined accessible period. Similarly, the One-Off Access Pattern generates a one-off, identifiable link for the verifier, which is a sub-pattern and a specific case of the Time-Constrained Access Pattern [11]. The essence of both is to provide a time-constrained link to the verifier, and can be used under different conditions, whether a long-term effective credential or an extremely short verification period. This pattern offers better flexibility as it can generate links with varying access periods that fulfill different conditions and requirements. To enhance privacy, both patterns ensure that the user account cannot be accessed maliciously after the time-out period, thereby preventing any further violation of private information with expired links. The access restriction mechanism objectively enhances system security. However, a snapshot may be maliciously created when access is available. Thus, the historical identity state can be leaked [11], which is an inherent constraint of this pattern. In addition, compared to the Selective Content Generation pattern, this may pose security risks. There are known real-world examples such as Snapchat[51] and Snappass[52] allow user-defined secret information to be deleted and cannot be recovered according to the user's setting.

Commonalities from DP67 to DP69: 1)All of these measures enhance privacy and security. User validation privacy is improved by minimizing exposed data or restricting access duration. As a result, private information can only be retrieved through a specific process. Security is also objectively improved due to the limitations, which reduce the attack surface and potential for malicious behavior. Despite the validation process being designed, it is impossible to completely eliminate attack possibility. In practice, there are still potential risks to consider and mitigate. 2)All of them lead to cost increases. Selective Content Generation (DP68) requires different credentials with different data. Meanwhile, Time-constrained Access (DP69) and One-off Access (DP70) require the generation and storage of verifiable links and the maintenance of temporary access mechanisms for specific durations. The process of extra content leads to additional communication complexity and system costs for blockchain.

C4.2: Business Cooperation

In conventional collaborations, one party cannot verify the activities or status of the others, and in low-trust environments, this can restrict business cooperation. But the decentralized environment of blockchain with its features of high availability, transparency, and traceability can mitigate this situation [70]. It enables business cooperation through

blockchain technology, even in unfamiliar environments. As a result, this section aims to collect and analyze design patterns that focus on entity interaction and cooperation in traditional business. As shwon in Fig. 13.

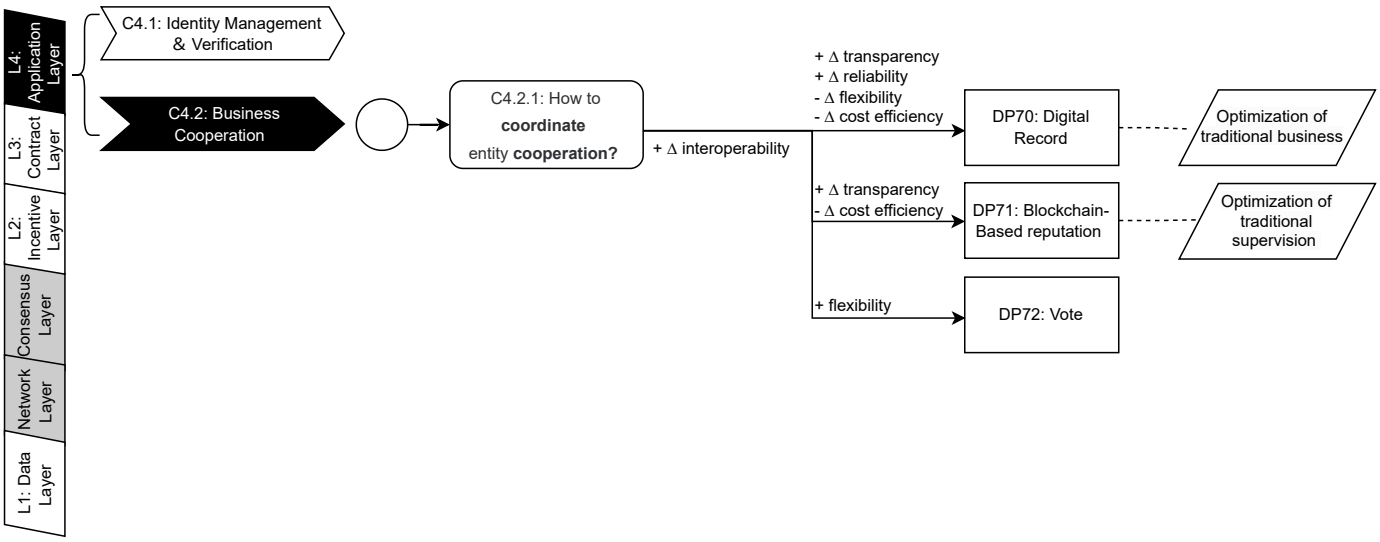

Fig. 13: Decision Model for Business Cooperation (C4.2)

TABLE 14: Quality Attribute Mappings of Blockchain-Based Design Patterns (Part C4.2: Business Cooperation)

ID	Pattern Name	Quality Attribute	Impact	Quality Attribute Mapping Description
70	Digital Record	Transparency	Positive	Stores business models and transactions on-chain. This allows any participant to verify data correctness at any time, ensuring transparency compared to traditional applications.
		Reliability	Positive	Blockchain's high-availability and immutable storage ensure reliability compared to traditional applications.
		Flexibility	Negative	Cannot perform time-streaming tasks well due to blockchain's verification and consensus mechanisms, leading less flexible compared with traditional applications.
		Cost Efficiency	Negative	Leads to increased computational and storage overhead due to the blockchain mechanism, compare with traditional applications.
71	Blockchain-based Reputation	Transparency	Positive	Facilitates transparent and credible cooperation between parties by enabling a decentralized reputation system accessible and maintained by all participants.
		Cost Efficiency	Negative	Requires additional storage and computing resources for maintaining the on-chain reputation mechanism.
72	Vote	Flexibility	Positive	Human intervention is allowed to address exceptions and resolve critical problems on-chain through voting, preventing the need for redeployment of the blockchain.
-	Commonality (DP70 to DP72)	Interoperability	Positive	All patterns improve interoperability compared with traditional systems, facilitating communication and collaboration between traditional entities and users off-chain.
		Transparency	Positive	Encourages storing data on-chain publicly to promote transparency and facilitate reliable collaboration among entities in low-trust environments.

C4.2.1: How to coordinate entity cooperation?

DP70: Digital Record. This pattern recommends storing business models and transactions on-chain for correctness verification at anytime by any participant, to keep data and process transparency [28]. As an inherent attribute, this pattern optimizes traditional business logic and realization rather than the blockchain itself. Therefore, the quality attributes discussed here are all compared with the traditional system. A decentralized system improves transparency of business processes. Also, with the high-available and immutable storage of blockchain, the system can be more reliable than traditional applications. However, this approach cannot perform time-streaming tasks well due to the verification and consensus mechanism of blockchain, and can only be executed in a triggered manner [12], which is less flexible. Additionally, some specific operations cannot be well-expressed through blockchain or lack a recognized way to record them [13]. Another concern is the increase in computational and storage overhead resulting from the large amount of data recorded on-chain. The ChromaWay solution is a realistic instance of this pattern in the blockchain record-keeping system [71].

DP71: Blockchain-based Reputation. This pattern employs blockchain to create a decentralized reputation system, enabling all participants to access and maintain reputation data for each entity. As an optimization of traditional systems,

this pattern addresses supervision problems and facilitates credible cooperation between parties [12]. Various variants of this pattern are proposed across different application domains, and a survey discussing these variants can be found in [72]. However, maintaining the on-chain reputation mechanism requires additional storage and computing resources. Furthermore, this pattern is limited to monitoring participants' reputation rather than ensuring business or data transparency. An instance of a business process that employs a reputation system is presented in [12].

DP72: Vote. This pattern allows participants to vote on a problem to reach consensus on certain values or states [5]. Different from the others, this pattern is used for enhancing the efficiency of blockchains. It utilizes tokens or voters' addresses for voting and can make decisions on crucial issues. This allows human intervention to address exceptions on-chain and resolve undefined problems. This feature prevents the need for redeployment of blockchain, resulting in improved flexibility. However, given the overhead and efficiency concerns, it only applies to critical problems that cannot be solved automatically on-chain or must be solved by voting. For example, users can vote on an upgrade of a decentralized application since there is no trusted authority [5]. As an example, in a contract called Dice[53], the owner may initiate a vote to determine if an emergency withdrawal is necessary.

Commonality from DP70 to DP72: With blockchain, all patterns improve interoperability compared with traditional systems. This facilitates communication and collaboration between traditional entities and users off-chain. These patterns encourage storing data on-chain publicly to promote transparency and facilitate sharing. The use of blockchain for storing business processes, reputation systems, and voting mechanisms allows reliable collaboration among entities in low-trust environments. These methods allow cooperation in data management and maintenance, and facilitate communication between different entities.

NOTES

[1]https://orisi.org/
[2]http://www.oraclize.it/
[3]https://lightning.network/
[4]https://blog.oraclize.it/encrypted-queries-private-data-on-a-public-blockchain-71d893fac2bf
[5]https://mlgblockchain.com/crypto-signature.html
[6]https://z.cash/
[7]https://medium.com/tomochain/tomochains-mainnet-launcheandetoken-swanning-schedule-6f556e2t77
[8]https://coindesk.com/3-billion-blockchain-tronkicks-off-token-migration-today
[9]https://vechain.com
[10]https://storj.io/blog/2017/04/token-migration-plan-pt.1/
[11]https://community.binance.org/topic/44/binance-chain-mainnet-swap
[12]https://medium.com/the-notice-board/bitizens-is-moving-to-tron-71e5c9a39ef
[13]https://medium.com/@karmaapp/karma-is-moving-from-eos-to-wax-b081100c2702
[14]https://kinecosystem.github.io/kin-ecosystem-sdk-docs/docs/migration_ios
[15]https://medium.com/telos-foundation/telos-token-distribution-use-of-the-eos-genesis-snapshot-why-2d849a2b0055
[16]https://medium.com/bithereum-network/bithereums-hard-spoon-snapshot-is-complete-c1814024ea9a
[17]https://medium.com/@gifto/mass-adoption-token-meets-mass-adoptionchain-gifto-migrates-to-binance-chain-af8cf906e13a
[18]https://kinecosystem.github.io/kin-ecosystem-sdk-docs/docs/migration_ios
[19]https://medium.com/@Qubicles/migrating-ethereum-qubicle-tokens-to-thetelos-chain-of-eos-io-using-the-eos21-protocol-e79c14fcf112
[20]https://forum.aeternity.com/t/frequentlyasked-questions-faq-token-migration-phases-1-2-3/1411
[21]https://steempeak.com/communityfork/@hiveio/announcing-thelaunch-of-hive-blockchain
[22]https://github.com/sheos-org/eos21
[23]https://interledger.org/interledger.pdf
[24]https://www.uport.me/
[25]https://trinity.iota.org/
[26]https://trezor.io/
[27]https://www.ledger.com/
[28]https://www.parity.io/
[29]https://cryptopp.com/
[30]https://github.com/ethereum/EIPs/blob/master/EIPS/eip-20.md
[31]https://digix.global/
[32]https://regis.nu/
[33]http://www.ethereum-alarm-clock.com/
[34]https://etherscan.io/address/0x684282178b1d61164febcf9609ca1-95bef9a33b5#code
[35]https://etherscan.io/address/0x5A5eFF38DA95b0D58b6C616f26-99168B480953C9#code
[36]https://github.com/OpenZeppelin/openzeppelin-solidity/blob/master/contracts/ownership/Ownable.sol
[37]https://raiden.network/
[38]https://en.bitcoin.it/wiki/Atomic_cross-chain_trading
[39]https://en.bitcoin.it/wiki/Multisignature
[40]https://github.com/ethereum/mist
[41]https://medium.com/coinmonks/gas-optimization-in-solidity-part-i-variables-9d5775e43dde
[42] https://chronobank.io/
[43] https://colony.io/
[44]https://ens.domains
[45] https://trufflesuite.com/
[46]https://regis.nu/
[47]https://www.cryptokitties.co/
[48]https://etherscan.io/address/0x54bda709fed875224eae569bb6817-d96ef7ed9ad#code
[49]https://cryptopunks.app/
[50]https://polkadot.network/
[51]https://www.snapchat.com/
[52]https://oneoffsecret.com/
[53]https://etherscan.io/address/0x2AB9f67A27f606272189b30705269-4D3a2B158bA#code

REFERENCES

[1] R. Mühlberger, S. Bachhofner, E. Castelló Ferrer, C. Di Ciccio, I. Weber, M. Wöhrer, and U. Zdun, "Foundational oracle patterns: Connecting blockchain to the off-chain world," in *Proceedings of the International Conference on Business Process Management*. Springer, 2020, pp. 35–51.

[2] A. W. Abreu and E. F. Coutinho, "A pattern adherence analysis to a blockchain web application," in *Proceedings of the International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2020, pp. 103–109.

[3] X. Xu, C. Pautasso, L. Zhu, Q. Lu, and I. Weber, "A pattern collection for blockchain-based applications," in *Proceedings of the 23rd European Conference on Pattern Languages of Programs*. ACM, 2018, pp. 1–20.

[4] V. Rajasekar, S. Sondhi, S. Saad, and S. Mohammed, "Emerging design patterns for blockchain applications," in *Proceedings of the 15th International Conference on Software Technologies*. ScitePress, 2020, pp. 242–249.

[5] C. R. Worley and A. Skjellum, "Opportunities, challenges, and future extensions for smart-contract design patterns," in *Proceedings of the International Workshops on Business Information Systems*. Springer, 2019, pp. 264–276.

[6] M. Wöhrer and U. Zdun, "Design patterns for smart contracts in the ethereum ecosystem," in *Proceedings of the International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*. IEEE, 2018, pp. 1513–1520.

[7] M. Wohrer and U. Zdun, "From domain-specific language to code: Smart contracts and the application of design patterns," *IEEE Software*, vol. 37, no. 5, pp. 37–42, 2020.

[8] M. Bartoletti and L. Pompianu, "An empirical analysis of smart contracts: Platforms, applications, and design patterns," in *Proceedings of the International Conference on Financial Cryptography and Data Security*. Springer, 2017, pp. 494–509.

[9] Z. Gao, Z. Zhuang, Y. Lin, L. Rui, Y. Yang, C. Zhao, and Z. Mo, "Select-storage: A new oracle design pattern on blockchain," in *Proceedings of the 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 2021, pp. 1177–1184.

[10] J. Eberhardt and S. Tai, "On or off the blockchain? insights on off-chaining computation and data," in *Proceedings of the 6th European Conference on Service-Oriented and Cloud Computing*. Springer, 2017, pp. 3–15.

[11] Y. Liu, Q. Lu, H.-Y. Paik, and X. Xu, "Design patterns for blockchain-based self-sovereign identity," in *Proceedings of the European Conference on Pattern Languages of Programs 2020*. ACM, 2020, pp. 1–14.

[12] M. Müller, N. Ostern, and M. Rosemann, "Silver bullet for all trust issues? blockchain-based trust patterns for collaborative business processes," in *Proceedings of the 18th International Conference on Business Process Management*. Springer, 2020, pp. 3–18.

[13] V. L. Lemieux, "A typology of blockchain recordkeeping solutions and some reflections on their implications for the future of archival preservation," in *Proceedings of the International Conference on Big Data*. IEEE, 2017, pp. 2271–2278.

[14] L. Marchesi, M. Marchesi, G. Destefanis, G. Barabino, and D. Tigano, "Design patterns for gas optimization in ethereum," in *Proceedings of the International Workshop on Blockchain Oriented Software Engineering (IWBOSE)*. IEEE, 2020, pp. 9–15.

[15] Q. Lu, X. Xu, Y. Liu, I. Weber, L. Zhu, and W. Zhang, "ubaas: A unified blockchain as a service platform," *Future Generation Computer Systems*, vol. 101, pp. 564–575, 2019.

[16] Q. Lu, X. Xu, Y. Liu, and W. Zhang, "Design pattern as a service for blockchain applications," in *Proceedings of the International Conference on Data Mining Workshops (ICDMW)*. IEEE, 2018, pp. 128–135.

[17] M. Schinle, C. Erler, A. R. Vetter, and W. Stork, "How to disclose selective information from permissioned dlt-based traceability systems?" in *Proceedings of the 2nd International Conference on Decentralized Applications and Infrastructures (DAPPS)*. IEEE, 2020, pp. 153–158.

[18] N. Six, C. N. Ribalta, N. Herbaut, and C. Salinesi, "A blockchain-based pattern for confidential and pseudo-anonymous contract enforcement," in *Proceedings of the 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 2020, pp. 1965–1970.

[19] H. D. Bandara, X. Xu, and I. Weber, "Patterns for blockchain data migration," in *Proceedings of the European Conference on Pattern Languages of Programs 2020*. ACM, 2020, pp. 1–19.

[20] N. Kannengießer, S. Lins, C. Sander, K. Winter, H. Frey, and A. Sunyaev, "Challenges and common solutions in smart contract development," *IEEE Transactions on Software Engineering*, vol. 48, no. 11, pp. 4291–4318, 2021.

[21] Y. Liu, Q. Lu, X. Xu, L. Zhu, and H. Yao, "Applying design patterns in smart contracts: A case study on a blockchain-based traceability application," in *Proceedings of the 1st International Conference on Blockchain–ICBC*. Springer, 2018, pp. 92–106.

[22] M. Zecchini, A. Bracciali, I. Chatzigiannakis, and A. Vitaletti, "On refining design patterns for smart contracts," in *Proceedings of the Euro-Par 2019 International Workshops on Parallel Processing*. Springer, 2020, pp. 228–239.

[23] P. Zhang, J. White, D. C. Schmidt, and G. Lenz, "Applying software patterns to address interoperability in blockchain-based healthcare apps," *arXiv:1706.03700*, 2017.

[24] P. Zhang, D. C. Schmidt, J. White, and G. Lenz, "Blockchain technology use cases in healthcare," in *Advances in computers*. Elsevier, 2018, vol. 111, pp. 1–41.

[25] P. Zhang, D. C. Schmidt, and J. White, "A pattern sequence for designing blockchain-based healthcare information technology systems," in *Proceedings of the 26th Conference on Pattern Languages of Programs*. ACM, 2021, pp. 1–22.

[26] A. Mavridou and A. Laszka, "Designing secure ethereum smart contracts: A finite state machine based approach," in *Proceedings of the 22nd International Conference on Financial Cryptography and Data Security*. Springer, 2018, pp. 523–540.

[27] M. Wohrer and U. Zdun, "Smart contracts: Security patterns in the ethereum ecosystem and solidity," in *Proceedings of the International Workshop on Blockchain Oriented Software Engineering (IWBOSE)*. IEEE, 2018, pp. 2–8.

[28] K. Saeedi, M. D. Almalki, D. Aljeaid, A. Visvizi, and M. A. Aslam, "Design pattern elicitation framework for proof of integrity in blockchain applications," *Sustainability*, vol. 12, no. 20, p. 8404, 2020.

[29] "IEEE standard for a software quality metrics methodology," *IEEE Std 1061-1998*, pp. 1–20, 1998.

[30] "IEEE standard for a software quality metrics methodology," *IEEE Std 1061-1992*, pp. 1–96, 1993.

[31] ISO/IEC, *ISO/IEC 9126. Software engineering – Product quality*. ISO/IEC, 2001.

[32] ISO/IEC 25010, *ISO/IEC 25010:2023, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*, Std., 2023.

[33] A. B. Bondi, "Characteristics of scalability and their impact on performance," in *Proceedings of the 2nd international workshop on Software and performance*, 2000, pp. 195–203.

[34] K. Barker, M. Askari, M. Banerjee, K. Ghazinour, B. Mackas, M. Majedi, S. Pun, and A. Williams, "A data privacy taxonomy," in *Dataspace: The Final Frontier: 26th British National Conference on Databases, BNCOD 26, Birmingham, UK, July 7-9, 2009. Proceedings 26*. Springer, 2009, pp. 42–54.

[35] S. De Capitani Di Vimercati, S. Foresti, G. Livraga, and P. Samarati, "Data privacy: Definitions and techniques," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 20, no. 06, pp. 793–817, 2012.

[36] X. Xu, H. D. Bandara, Q. Lu, I. Weber, L. Bass, and L. Zhu, "A decision model for choosing patterns in blockchain-based applications," in *Proceedings of the 18th International Conference on Software Architecture (ICSA)*. IEEE, 2021, pp. 47–57.

[37] X. Sun, F. R. Yu, P. Zhang, Z. Sun, W. Xie, and X. Peng, "A survey on zero-knowledge proof in blockchain," *IEEE network*, vol. 35, no. 4, pp. 198–205, 2021.

[38] X. Xu, C. Pautasso, L. Zhu, V. Gramoli, A. Ponomarev, A. B. Tran, and S. Chen, "The blockchain as a software connector," in *Proceedings of the 13th Working IEEE/IFIP Conference on Software Architecture (WICSA)*. IEEE, 2016, pp. 182–191.

[39] H. Al-Breiki, M. H. U. Rehman, K. Salah, and D. Svetinovic, "Trustworthy blockchain oracles: Review, comparison, and open research challenges," *IEEE Access*, vol. 8, pp. 85 675–85 685, 2020.

[40] S. Goswami, S. M. Danish, and K. Zhang, "Towards a middleware design for efficient blockchain oracles selection," in *Proceedings of the 4th International Conference on Blockchain Computing and Applications (BCCA)*, 2022, pp. 55–62.

[41] S. Ellis, A. Juels, and S. Nazarov, "Chainlink: A decentralized oracle network," *Retrieved March*, vol. 11, no. 2018, p. 1, 2017.

[42] I. Weber, X. Xu, R. Riveret, G. Governatori, A. Ponomarev, and J. Mendling, "Untrusted business process monitoring and execution using blockchain," in *Proceedings of the 14th International Conference on Business Process Management*. Springer, 2016, pp. 329–347.

[43] J. Heiss, J. Eberhardt, and S. Tai, "From oracles to trustworthy data on-chaining systems," in *Proceedings of the 2019 IEEE International Conference on Blockchain (Blockchain)*. IEEE, 2019, pp. 496–503.

[44] J. Eberhardt and J. Heiss, "Off-chaining models and approaches to off-chain computations," in *Proceedings of the 2nd Workshop on Scalable and Resilient Infrastructures for Distributed Ledgers*. ACM, 2018, pp. 7–12.

[45] N. Neidhardt, C. Köhler, and M. Nüttgens, "Cloud service billing and service level agreement monitoring based on blockchai." in *Proceedings of the 9th International Workshop on Enterprise Modeling and Information Systems Architectures*. CEUR-WS.org, 2018, pp. 65–69.

[46] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi, "Town crier: An authenticated data feed for smart contracts," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2016, pp. 270–282.

[47] T. Hepp, M. Sharinghousen, P. Ehret, A. Schoenhals, and B. Gipp, "On-chain vs. off-chain storage for supply-and blockchain integration," *it-Information Technology*, vol. 60, no. 5-6, pp. 283–291, 2018.

[48] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou, "Hawk: The blockchain model of cryptography and privacy-preserving smart contracts," in *Proceedings of the 2016 IEEE symposium on security and privacy (SP)*. IEEE, 2016, pp. 839–858.

[49] J. Benet, "Ipfs-content addressed, versioned, p2p file system," *arXiv:1407.3561*, 2014.

[50] V. Trón, A. Fischer, D. A. Nagy, Z. Felföldi, and N. Johnson, "Swap, swear and swindle: Incentive system for swarm," *Ethereum Orange Paper*, 2016.

[51] K. Wüst and A. Gervais, "Do you need a blockchain?" in *Proceedings of the 2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*. IEEE, 2018, pp. 45–54.

[52] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, "From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again," in *Proceedings of the 3rd Science Conference on Innovations in Theoretical Computer*. ACM, 2012, pp. 326–349.

[53] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," *Communications of the ACM*, vol. 59, no. 2, pp. 103–112, 2016.

[54] E. B. Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, "Zerocash: Decentralized anonymous payments from bitcoin," in *Proceedings of the International Conference on Symposium on Security and Privacy*. IEEE, 2014, pp. 459–474.

[55] B. de Azevedo Mendonça and P. Matias, "Auditchain: A mechanism for ensuring logs integrity based on proof of existence in a public blockchain," in *Proceedings of the 11th IFIP International Conference on New Technologies, Mobility and Security (NTMS)*. IEEE, 2021, pp. 1–5.

[56] L. Owens, B. Razet, W. B. Smith, and T. C. Tanner, "Inter-family communication in hyperledger sawtooth and its application to a crypto-asset framework," in *Proceedings of the 15th International Conference on Business Information Systems Distributed Computing and Internet Technology*. Springer, 2019, pp. 389–401.

[57] N. Six, N. Herbaut, and C. Salinesi, "Blockchain software patterns for the design of decentralized applications: A systematic literature review," *Blockchain: Research and Applications*, vol. 3, no. 2, p. 100061, 2022.

[58] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, "Towards scaling blockchain systems via sharding," in *Proceedings of the International Conference on Management of Data*. ACM, 2019, pp. 123–140.

[59] A. R. Sai, J. Buckley, and A. Le Gear, "Assessing the security implication of bitcoin exchange rates," *Computers & Security*, vol. 86, pp. 206–222, 2019.

[60] Y. Chen, "Blockchain tokens and the potential democratization of entrepreneurship and innovation," *Business horizons*, vol. 61, no. 4, pp. 567–575, 2018.

[61] J. Huang, K. Lei, M. Du, H. Zhao, H. Liu, J. Liu, and Z. Qi, "Survey on blockchain incentive mechanism," in *Proceedings of the 5th International Conference of Pioneering Computer Scientists, Engineers and Educators*. Springer, 2019, pp. 386–395.

[62] Q. Lu and X. Xu, "Adaptable blockchain-based systems: A case study for product traceability," *Ieee Software*, vol. 34, no. 6, pp. 21–27, 2017.

[63] J. Chen, X. Xia, D. Lo, J. Grundy, X. Luo, and T. Chen, "Defining smart contract defects on ethereum," *IEEE Transactions on Software Engineering*, vol. 48, no. 1, pp. 327–345, 2020.

[64] K. Bhargavan, A. Delignat-Lavaud, C. Fournet, A. Gollamudi, G. Gonthier, N. Kobeissi, N. Kulatova, A. Rastogi, T. Sibut-Pinote, N. Swamy *et al.*, "Formal verification of smart contracts: Short paper," in *Proceedings of the 2016 ACM Workshop on Programming Languages and Analysis for Security*. ACM, 2016, pp. 91–96.

[65] M. Kaleem, A. Mavridou, and A. Laszka, "Vyper: A security comparison with solidity based on common vulnerabilities," in *Proceedings of the 2nd International Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*. IEEE, 2020, pp. 107–111.

[66] M. Rodler, W. Li, G. O. Karame, and L. Davi, "Sereum: Protecting existing smart contracts against re-entrancy attacks," *arXiv preprint arXiv:1812.05934*, 2018.

[67] N. Grech, M. Kong, A. Jurisevic, L. Brent, B. Scholz, and Y. Smaragdakis, "Madmax: Surviving out-of-gas conditions in ethereum smart contracts," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–27, 2018.

[68] M. Wöhrer and U. Zdun, "Domain specific language for smart contract development," in *Proceedings of the International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2020, pp. 1–9.

[69] S. Amani, M. Bégel, M. Bortin, and M. Staples, "Towards verifying ethereum smart contract bytecode in isabelle/hol," in *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, 2018, pp. 66–77.

[70] D. Virovets and S. Obushnyi, "Decentralized autonomous organizations as the new form of economic cooperation in digital world," *The USV Annals of Economics and Public Administration*, vol. 20, no. 2 (32), pp. 41–52, 2021.

[71] V. L. Lemieux, "Evaluating the use of blockchain in land transactions: An archival science perspective," *European property law journal*, vol. 6, no. 3, pp. 392–440, 2017.

[72] E. Bellini, Y. Iraqi, and E. Damiani, "Blockchain-based distributed trust and reputation management systems: A survey," *IEEE Access*, vol. 8, pp. 21 127–21 151, 2020.